



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV AUTOMATIZACE A MĚŘICÍ TECHNIKY

DEPARTMENT OF CONTROL AND INSTRUMENTATION

DEMONSTRAČNÍ LABORATORNÍ ÚLOHA V JAZYCE SFC

DEMONSTRATION LAB TASK IN THE SFC LANGUAGE

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Eduard Sysel

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Radek Štohl, Ph.D.

BRNO 2025



Bakalářská práce

bakalářský studijní program **Automatizační a měřicí technika**

Ústav automatizace a měřicí techniky

Student: Eduard Sysel

ID: 240439

Ročník: 3

Akademický rok: 2024/25

NÁZEV TÉMATU:

Demonstrační laboratorní úloha v jazyce SFC

POKYNY PRO VYPRACOVÁNÍ:

Cílem práce je vytvořit demonstrační úlohu do výuky programovatelných automatů v jazyce SFC.

- 1) Provedte literární rešerši o programovacích jazycích pro PLC/PAC.
- 2) Popište programovací jazyk Sequential Function Charts pro řadu LOGIX.
- 3) Navrhněte demonstrační laboratorní úlohu.
- 4) Vytvořte program a vizualizaci pro demonstraci vlastností SFC.
- 5) Ověřte své řešení.

DOPORUČENÁ LITERATURA:

Logix 5000 Controllers. General Instructions. Reference Manual. 2023. 1756-RM003Y-EN-P. Rockwell Automation Technologies, Inc. USA, 861 stran.

Logix 5000 Controllers. Sequential Function Charts. Programming Manual. 2022. 1756-PM006K-EN-P. Rockwell Automation Technologies, Inc. USA, 81 stran.

Termín zadání: 10.2.2025

Termín odevzdání: 28.5.2025

Vedoucí práce: Ing. Radek Štohl, Ph.D.

Ing. Miroslav Jirgl, Ph.D.
předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č.40/2009 Sb.

ABSTRAKT

Tato práce se zabývá programovacím jazykem SFC a jeho praktickou demonstrací na laboratorní úloze. V úvodní části jsou popsány základní prvky jazyka SFC a jejich použití na konkrétních příkladech. Zvláštní pozornost je věnována kvalifikátorům akcí, které představují klíčový prvek pro pochopení programování v tomto jazyce.

Práce dále popisuje demonstrační úlohu vytvořenou pro fyzický model „Kuličky“, na kterém je implementace v jazyce SFC prakticky ověřena. Program v jazyce SFC je porovnán s implementací v jazycích LAD a ST, přičemž jsou diskutovány výhody a nevýhody jednotlivých přístupů a situace, kdy je vhodné zvolit daný jazyk. Součástí řešení je také vizualizace v prostředí HMI, která umožňuje uživateli sledovat kroky programu v diagramu.

KLÍČOVÁ SLOVA

SFC, sekvenční řízení, PLC, programování PLC, akční kvalifikátory, grafické programování, vizualizace, HMI, jazyk LAD, jazyk ST, průmyslová automatizace,

ABSTRACT

This thesis focuses on the SFC programming language and its practical demonstration through a laboratory assignment. The introductory section outlines the fundamental elements of the SFC language and demonstrates their usage with specific examples. Particular emphasis is placed on action qualifiers, which are a key concept for understanding the structure and logic of programming in SFC.

The thesis further presents a demonstration task developed for a physical model called “Balls,” where the SFC implementation is practically verified. The SFC-based program is compared with implementations in LAD and ST languages, discussing the benefits and limitations of each approach, as well as the scenarios in which a particular language is more suitable. The solution also includes an HMI visualization that enables the user to monitor each program step through a graphical diagram.

KEYWORDS

SFC, sequential control, PLC, PLC programming, action qualifiers, graphical programming, visualization, HMI, LAD language, ST language, industrial automation

SYSEL, Eduard. *Demonstrační laboratorní úloha v jazyce SFC*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav automatizace a měřicí techniky, 2025. Vedoucí práce: Ing. Radek Štohl, Ph.D.

Prohlášení autora o původnosti díla

Jméno a příjmení autora: Eduard Sysel
VUT ID autora: 240439
Typ práce: Bakalářská práce
Akademický rok: 2025/05
Téma závěrečné práce: Demonstrační laboratorní úloha v jazyce SFC

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno
.....
podpis autora*

* Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu své bakalářské práce, panu Ing. Radku Štohlovi, Ph.D. za odborné vedení, cenné rady a ochotu při konzultacích.

Obsah

Úvod	12
1 Popis programovacích jazyků pro PLC	13
1.1 Programovatelné řídicí systémy a jejich jazyky	13
1.1.1 LAD - Ladder Diagram	13
1.1.2 ST - Structured Text	14
1.1.3 FBD - Function Block Diagram	14
1.1.4 IL - Instruction List	14
1.1.5 SFC - Sequential Function Chart	15
1.1.6 Srovnání možností a použití jazyků	15
1.2 Úvod do jazyka SFC	16
1.2.1 Popis základních bloků a principů	18
1.2.2 Tok signálů v SFC	20
2 Jazyk SFC pro řadu LOGIX	23
2.1 Struktura SFC_STEP	23
2.1.1 Step.(X, FS, LS, SA)	23
2.1.2 Step.(T, PRE, TMAX)	24
2.1.3 Step.(AlarmEn, LimitLow, LimitHigh)	24
2.1.4 Step.(AlarmLow, AlarmHigh)	24
2.1.5 Step.(OV, Count, Status)	24
2.2 Struktura SFC_ACTION	26
2.2.1 Action.(Q, A)	26
2.2.2 Action.(T, PRE, PauseTimer)	26
2.2.3 Action.(Count, Status)	26
2.2.4 Boolean a Non-Boolean akce	27
2.2.5 Posloupnost vykonání akcí v kroku	27
2.3 Kvalifikátory akce	27
2.3.1 N: Non-stored	28
2.3.2 S: Stored	28
2.3.3 R: Reset	28
2.3.4 Porovnání N, S, R	28
2.3.5 Kvalifikátory P, P0, P1	30
2.3.6 Porovnání P, P0, P1	30
2.3.7 Kvalifikátory L, D	32
2.3.8 Kvalifikátory SD, SL, DS	33
2.3.9 Rozdíl mezi SD a DS	34

2.4	Použití časovače v jazyku SFC	36
2.5	Subrutiny	36
2.6	Ukončení SFC rutiny	37
2.7	Dokumentace SFC: Nastavení tisku rutin	37
3	Demonstrační úloha	38
3.1	Popis demonstrační úlohy: Kuličky	38
3.2	Programování demonstrační úlohy	41
3.2.1	Přehled a architektura programu	41
3.2.2	MainProgram: SFC_MainRoutine	42
3.2.3	MainProgram: DávkováníL,DávkováníM a DávkováníR	45
3.2.4	Konec programu a resetování programu na první krok	46
3.2.5	BCD_PrevodProgram	50
3.2.6	AlarmsProgram	51
3.2.7	Informace_HMI_Logic	51
3.2.8	TlacitkaSetupProgram	51
3.2.9	Porovnání hlavní rutiny v jazyce LAD	53
3.3	Vizualizace demonstrační úlohy	54
3.3.1	Obrazovka Ovládání	55
3.3.2	Obrazovka StopRutina	57
3.3.3	Obrazovka Informace_HMI	57
	Závěr	59
	Literatura	60
	A Fotografie demostračního modelu: Kuličky	61
	B Obsah elektronické přílohy	62

Seznam obrázků

1.1	Ukázka jazyku LAD	14
1.2	Ukázka jazyku ST	14
1.3	Ukázka jazyku FBD	15
1.4	Ukázka jazyku IL [4]	15
1.5	Základní bloky SFC	18
1.6	SFC jako vývojový diagram	20
1.7	Paralelní větev	21
1.8	Výběrová větev	21
1.9	Paralelní a výběrové větve [6]	21
1.10	Inicializační krok.	21
2.1	Struktura Stepu v Studiu 5000	23
2.2	Struktura Akce v Studiu 5000	26
2.3	Porovnání N s S kvalifikátorem	28
2.4	Porovnání S s N kvalifikátorem	29
2.5	Resetování akce s kvalifikátorem S	30
2.6	Porovnání P0 a P1	31
2.7	Kvalifikátor P	32
2.8	Porovnání L a D	33
2.9	Porovnání SD a DS	34
2.10	Využití interního časovače struktury SFC_STEP	36
2.11	Použití FBD funkce časovače v akci pomocí jazyku ST	37
3.1	Vizuální reprezentace laboratorního modelu: Kuličky [Příloha A]	38
3.2	Postup uživatele při průchodu dávkování	40
3.3	Diagram hlavní rutiny	42
3.4	Step_Init a Step_Ready	43
3.5	Step_ZLED	44
3.6	Step_Davkovani a Step_DavkovaniComplete	45
3.7	Rutina dávkování pro levý válec – část 1/2	46
3.8	Rutina dávkování pro levý válec – část 2/2	47
3.9	Rutina DetekceStop	47
3.10	Rutina pro stav stop - StopRoutine	48
3.11	Hlavní rutina a její paralelní větev pro ukončení programu	49
3.12	Rutina pro konverzi BCD kódu na DINT číslo	50
3.13	Detekce hodnoty nižší než nula pro Válec M	51
3.14	Logika zobrazení informačního okna pro ukončení programu	52
3.15	Logika proměnných Start_tlacitko a Stop_tlacitko	52
3.16	Porovnání kroků Step_ZLED a Step_ZLED2 v SFC s jazykem LAD	53

3.17 Krok Step_Davkovani	53
3.18 Krok Step_Davkovani v SFC v jazyku LAD	54
3.19 Diagram obrazovek vizualizace	54
3.20 Obrazovka ovládání s bez animací	55
3.21 Obrazovka ovládání bez varovných indikátorů	56
3.22 Obrazovka ovládání po spuštění dávkování	57
3.23 Obrazovka stop rutiny	58
3.24 Informační obrazovka	58
A.1 Demonstrační model: Kuličky	61

Seznam tabulek

2.1	Popis základních tagů kroku v SFC	25
2.2	Popis základních tagů pro akce	27
2.3	Popis kvalifikátorů pro akce	35
3.1	Vstupy a výstupy modulu a jejich zápis do PLC tagů	39

Úvod

Tato bakalářská práce spadá do oblasti průmyslové automatizace, konkrétně návrhem a programováním řídicího systému typu PLC s využitím programovacího jazyka SFC (Sequential Function Chart). Cílem práce je demonstrovat schopnosti jazyku SFC pro řadu kontrolérů LOGIX.

V prvních kapitolách této práce jsou popsány programovací jazyky pro PLC, standardizováno podle normy IEC 61131-3. Jazyk SFC je popsán podrobně v samostatné kapitole ve formě manuálu. Další část práce se zabývá demonstrační úlohou s názvem „Kuličky“. Tato úloha byla vytvořena pro laboratorní modul, jehož úkolem je počítat kuličky ve třech válcích. Počet kuliček, které má modul spočítat, určuje uživatel pomocí číslicových voličů pro každý válec. Program je naprogramovaný celý v SFC a využívá většinu funkcí, které SFC nabízí.

V další části práce se poté porovnává hlavní rutina programu se stejnými rutinami v jazyku LAD, kde jsou diskutovány možnosti použití jazyku SFC. Pro demonstrační úlohu byla vytvořena vizualizace, která zobrazuje průběh vykonávání jednotlivých rutin programu pomocí jejich diagramů v HMI.

1 Popis programovacích jazyků pro PLC

Tato část práce se bude zabývat základními pojmy souvisejícími s programováním PLC. V prvních kapitolách jsou porovnány ostatní programovací jazyky pro PLC s SFC, aby čtenář mohl posoudit, zda je právě jazyk SFC vhodný pro jeho aplikaci. Následně je práce zaměřena na základní principy a strukturu jazyka SFC.

Popis jazyka SFC bude uveden na příkladech kódu, které budou sloužit jako studijní podklad v oblasti programování PLC.

1.1 Programovatelné řídicí systémy a jejich jazyky

Programování systémů PLC (Programmable Logic Controller) a PAC (Programmable Automation Controller) tvoří klíčovou roli v automatizaci a řízení průmyslových procesů. Tyto systémy umožňují efektivní a spolehlivé řízení strojů a zařízení v různých aplikacích, od výrobních linek po komplexní automatizační systémy.

Programovatelné logické automaty (PLC) jsou průmyslové počítače určené k řízení procesů, které díky snadnému programování, vysoké spolehlivosti a kompaktní velikosti nahradily složitou kabeláž reléových systémů. Moderní PLC umožňují rychlé úpravy, pokročilé funkce, jako je zpracování dat či řízení pohybu, a jsou klíčové pro flexibilitu a efektivitu průmyslové automatizace.

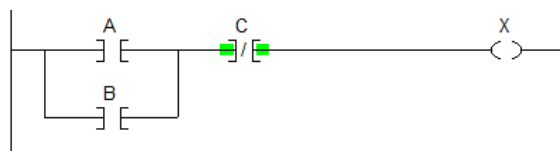
Programovatelné automační kontroléry (PAC) jsou výkonnější než PLC, nabízejí širší možnosti zpracování dat, pokročilé řízení procesů a integrované funkce pro práci s komplexními systémy, jako je strojové vidění či IoT. Na rozdíl od PLC jsou PAC optimalizovány pro složité aplikace s vysokými nároky na výkon, škálovatelnost a flexibilní komunikaci mezi zařízeními.[2][1]

Norma IEC 61131-3 definuje pět programovacích jazyků. Pro každý z těchto jazyků je uvedeno, jak se vytváří logická funkce AND.

1.1.1 LAD - Ladder Diagram

Tento jazyk připomíná elektrické schéma a používá grafické prvky. Skládá se z networků (lze představit jako řádky v kódu), do kterých vkládáme kontakty (vstupy) a cívky (výstupy). Kromě těchto dvou základních prvků má LD knihovnu různých bloků, které uživatel může vložit, například časovače nebo čítače.

Při spuštění programu jde pozorovat tok signálů a to usnadňuje diagnostiku a údržbu automatizačních systémů.



Obr. 1.1: Ukázka jazyku LAD

1.1.2 ST - Structured Text

Tento jazyk poskytuje textovou syntaxi, která umožňuje psaní složitějších algoritmů a manipulaci s daty, podobně jako v tradičních programovacích jazycích. Uživatelé mají možnost definovat proměnné, funkce a procedury, čímž se rozšiřují možnosti programování. ST je ideální pro úkoly, které vyžadují složité výpočty, cykly a podmíněné příkazy. Jeho strukturální formát usnadňuje čitelnost a údržbu kódu, což může být výhodou při práci na rozsáhlých projektech.[1]

```

1  IF ( A OR B ) AND NOT C THEN
2      X := 1;
3  ELSE
4      X := 0;
5  END_IF;
  
```

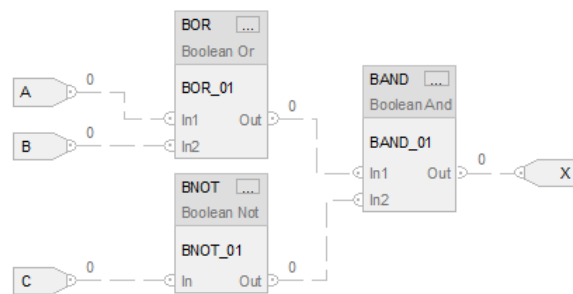
Obr. 1.2: Ukázka jazyku ST

1.1.3 FBD - Function Block Diagram

Programování ve funkčních blocích používá instrukce, které jsou naprogramovány jako sled vzájemně propojených bloků. Typické typy funkčních bloků jsou kombinační, sekvenční bloky, časovače a čítače. Schémata funkčních bloků jsou svým uspořádáním podobná elektrickým blokovým schémátům. Primárním konceptem je datový tok. Funkční bloky jsou vzájemně propojeny a tvoří obvod, který splňuje požadavky pro ovládání. Datový tok od vstupů, přes funkční bloky nebo instrukce a poté k výstupům. Použití funkčních bloků pro programování PLC získává stále širší použití. [1]

1.1.4 IL - Instruction List

Tento jazyk je nízkoúrovňový a připomíná assembler, protože se skládá z jednoduchých příkazů, které se provádějí postupně. IL je určen pro specifické aplikace, kde je vyžadována maximální kontrola nad hardwarem a výkonem. Ačkoliv je efektivní



Obr. 1.3: Ukázka jazyku FBD

pro určité úkoly, může být pro některé uživatele méně čitelný a náročnější na učení. Důvodem je jeho syntaxe, která neobsahuje vizuální reprezentaci, což může ztížit diagnostiku a údržbu kódu v porovnání s grafickými jazyky. Ve třetím vydání normy IEC 61131-3 byla ukončena podpora tohoto jazyka.[3][4]

LD	Speed
GT	2000
JMPCN	VOLTS_OK
LD	Volts
VOLTS_OK LD	1
ST	%Q75

Obr. 1.4: Ukázka jazyku IL [4]

1.1.5 SFC - Sequential Function Chart

Tento jazyk je navržen pro modelování sekvenčního řízení a zobrazuje posloupnost kroků a přechodů mezi nimi. Lze ho přirovnat k vývojovému diagramu. Uživatelé mohou definovat akce, které aktivují přechody. SFC usnadňuje organizaci logiky v aplikacích, kde je důležitá sekvenční činnost, jako je výrobní linka nebo procesní kontrola. Díky své vizuální povaze je SFC užitečný pro rychlé schématické návrhy a přehledné sledování kroků v procesu. Podrobnější popis SFC viz.kapitola 2. [5][6]

1.1.6 Srovnání možností a použití jazyků

Obecně lze každý z uvedených programovacích jazyků považovat za přehledný, avšak záleží na konkrétním způsobu použití a typu řešeného úkolu. Uvažujme například krátký program v jazyce LAD, který obsahuje pouze několik networků (sítí) a jednoduché logické operace. Díky toku signálu bude přehledně viditelné, které vstupy jsou

aktivovány a jaké výstupy z nich vyplývají. Pokud bychom zvolili podobný příklad v jazyce ST nebo v jiných programovacích jazycích, kód by sice zůstal přehledný, avšak v LAD by byl výsledek jasně patrný. Na druhou stranu, pokud bychom chtěli v LAD implementovat složitý kód, zahrnující komplexní algoritmy, stal by se tento jazyk poměrně nepřehledným, protože by byl program roztažený na spoustu networků. Podobný problém by nastal i pro SFC. V takovém případě je výhodnější využít jazyk ST, který umožňuje kompaktní zápis složitých algoritmů, zatímco ostatní jazyky by byly značně roztáhlé a tudíž nepřehledné.

Můžeme se rovněž setkat se situací, kdy je třeba provádět úlohu sekvenčně. Zde je vhodným kandidátem jazyk SFC, ale mnozí PLC programátoři by přesto upřednostnili jazyk LAD, jelikož i ten lze psát sekvenčně. Kdy tedy použít SFC?

SFC je vhodné využít ve spojení s ostatními jazyky, například jako subrutinu hlavního programu. Uplatní se zejména v krátkých sekvenčních cyklech, které se označují jako batch processing. Tento přístup lze využít například pro konkrétní úkon na výrobní lince, jako je řízení pohybu vrtáku, kdy každý krok (step) našeho podprogramu v SFC by představoval jeden pohyb vrtáku. Podobně to může být i naopak. Celý hlavní program bude napsán v SFC a každý krok by představoval větší část výrobní linky. Takto by například celá linka mohla být rozdělena do šesti kroků (stepů v SFC), přičemž každý krok by byl podprogramem zajišťujícím konkrétní činnost na lince.

Výhodné použití SFC může tedy být jako přehledný podprogram pro krátké opakující se procesy nebo jako hlavní rutina, pomocí které jsou spouštěny jednotlivé složitější podkroky. Samozřejmě ho lze použít i pro delší a komplexnější úkoly, avšak v případě, kdy program udržuje osoba, která se nepodílela na jeho vývoji, by mohl být takový komplexní SFC nepřehledný.

1.2 Úvod do jazyka SFC

Jazyk byl vyvinut až po zavedení jazyků LD, ST a diagramů logických bran. Tyto starší jazyky byly poměrně složité, a pro uživatele s jen základními znalostmi bylo obtížné pochopit jednotlivé kroky a jejich vliv na celý program. Bylo tedy nutné vytvořit jazyk, který by byl snadno naučitelný a pro laika jednoduše pochopitelný.

V roce 1979 byla vytvořena první verze SFC pod názvem GRAFCET. Tato verze vznikla v Evropě díky francouzské asociaci pro ekonomii a aplikovanou kybernetiku. GRAFCET je akronymem pro „GRAPhe Fonctionnel de Commande Étape Transition“, což se v angličtině překládá jako Step Transition Function Charts. Jedná se o grafickou metodu s jednoduchou syntaxí a přehlednou grafickou vizualizací.

V první polovině 80. let jeho popularita rostla a v roce 1993 byl při vytváření normy IEC 61131 kromě LD, ST, FBD a IL přidán také GRAFCET, tentokrát pod

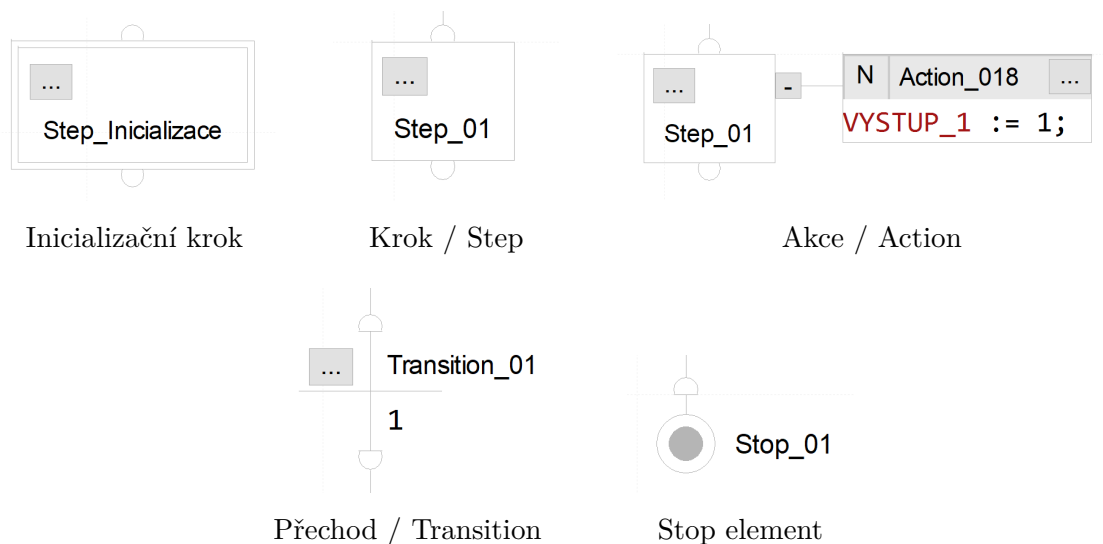
názvem Sequential Function Chart, neboli SFC.

Jazyk SFC, známý jako sekvenční funkční diagram, si lze představit jako vývojový diagram upravený pro programování PLC. Patří mezi často používané jazyky, zejména pro aplikace v dávkovém zpracování (batch processing).

Dávkové zpracování je definováno jako konečný cyklus, ve kterém má každý krok provádět specifické úkony. Každý krok musí začít až po dokončení předchozího kroku. Cyklus dávky může být podle potřeby dynamicky upraven v závislosti na ostatních procesech. Tento přístup se typicky využívá v průmyslových odvětvích, jako je potravinářství, chemie, farmacie nebo pivovarnictví, kde procesy probíhají v diskrétních množstvích a mají pevně definované kroky (např. míchání, ohřívání, chlazení). [7][8]

1.2.1 Popis základních bloků a principů

Jazyk SFC slouží k vytváření sekvenčních programů a je přehledný pro zobrazení kroků a přechodů v procesech. Program v SFC se skládá ze základních prvků, jako jsou inicializační krok, krok a přechod. Pro funkční program postačují pouze tyto tři prvky, avšak ve většině programů se zpravidla používají i bloky Akce a Stop elementů.



Obr. 1.5: Základní bloky SFC

Step_Inicializace

Každý SFC musí začínat právě tímto blokem. Tento blok je v podstatě identický s klasickým krokem, jediným rozdílem je, že inicializační krok může být v programu použit pouze jednou, protože určuje překladači, kde naše rutina začíná. Inicializační krok můžeme vytvořit ve vlastnostech normálního kroku.[6]

Step

Krok je základním stavebním kamenem jazyka SFC. Lze jej přirovnat k menšímu networku v jazyce LD. Sám o sobě není schopen vykonávat kód, je nutné do kroku přiřadit akci, do které následně vložíme náš kód v jazyce ST.

Každý vytvořený krok má několik tagů, které může uživatel využít. Pomocí těchto tagů lze zjistit, zda je krok aktivní nebo jak dlouho je aktivní. Podrobný popis těchto funkcí bude uveden v kapitole 2.1.

Akce

Akce nemůže existovat samostatně; musí být vždy přiřazena ke kroku. Umožňuje zapisovat kód, který se má vykonat v kroku, ke kterému je přiřazena. V jednom kroku může být přiřazeno libovolné množství akcí, které se vykonávají postupně shora dolů.

Podobně jako krok má každá akce své vlastní tagy, které může uživatel využít. Oproti kroku má akce navíc kvalifikátor, který určuje, kdy má být akce aktivní. Podrobnější popis kvalifikátorů je uveden v kapitole 2.2. Celkově existuje 11 kvalifikátorů, ale prozatím budeme pracovat s kvalifikátorem N. Tento kvalifikátor říká, že akce má být prováděna nepřetržitě, dokud krok, ke kterému je akce přiřazena, nepřestane být aktivní. Kvalifikátor se nachází v levém horním rohu akce.

Transition

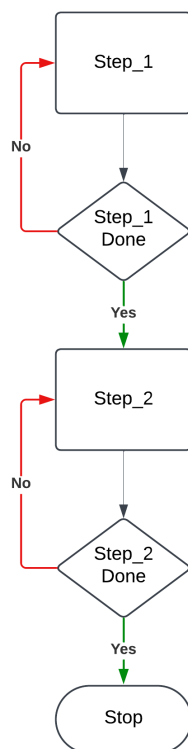
Slouží jako přechod mezi jednotlivými kroky v kódu. Do přechodu lze zapsat podmínku pro jeho aktivaci v jazyce ST. Pokud je podmínka nastavena na hodnotu 1, znamená to, že přechod bude vždy platný. Na rozdíl od ostatních bloků nemá přechod (transition) své vlastní tagy.

Stop element

Blok, který ukončí rutinu ve chvíli, kdy se stane aktivním. Na rozdíl od inicializačního kroku není jeho použití povinné. Na konci programu lze ponechat prázdný krok (step) nebo cyklit program zpět na jeho začátek. Tento blok rovněž má vnitřní tagy, které slouží pro práci se Stop elementem.

1.2.2 Tok signálů v SFC

Na rozdíl od jiných programovacích jazyků je SFC navržen speciálně pro sekvenční řízení. Lze jej přirovnat jako komplexnější vývojový diagram. Řízení postupuje shora dolů jednotlivými kroky, které jsou odděleny podmínkami. SFC je vyzobrazen ve formě vývojového diagramu (viz obrázek 1.6). Na rozdíl od vývojového diagramu



Obr. 1.6: SFC jako vývojový diagram

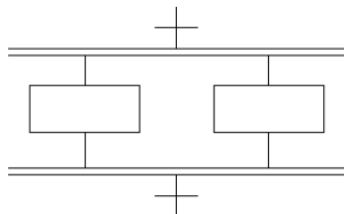
musí být v SFC mezi jednotlivými bloky vždy definovány podmínky. Po splnění podmínky se program přesune do následujícího kroku, zatímco předchozí krok se stane neaktivním. Pokud bychom měli v kódu pouze sekvenční větve, tak v daném okamžiku může být aktivní pouze jeden krok.

Klíčový rozdíl jsou akce, které ale nelze dobře zobrazit v klasickém vývojovém diagramu a více o nich v kapitole 2.2.

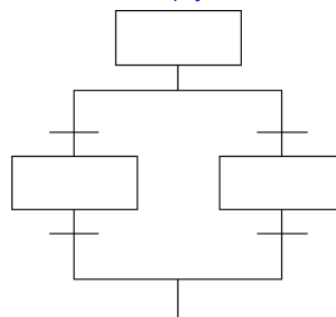
Paralelní a výběrové kroky

SFC nemusí mít vždy pouze sekvenční strukturu. Pokud bychom potřebovali, aby bylo několik kroků aktivních můžeme vytvořit paralelní větve. Paralelní větev se skládá ze dvou nebo více kroků které mají společnou počáteční a koncovou podmínku. Uživatel může vytvořit paralelní kód různými způsoby, avšak jeho struktura

nemusí být vždy přehledná. Paralelní větev je však srozumitelná i pro laika. Kromě paralelní větve lze vytvořit také selektivní větev, která umožňuje rozhodování mezi více možnými kroky. Každý krok má svou vstupní i výstupní podmínku, a na základě toho, která podmínka je splněna, se vybere odpovídající cesta.



Obr. 1.7: Paralelní větev

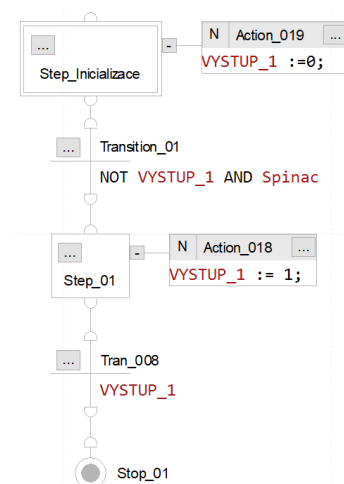


Obr. 1.8: Výběrová větev

Obr. 1.9: Paralelní a výběrové větve [6]

Demonstrace základních bloků

S využitím těchto základních bloků lze vytvořit jednoduchý program, jehož jediným účelem je zapsat logickou hodnotu 1 na výstup. Kód by bylo možné zjednodušit, aby byl kratší, avšak je důležité dodržovat určitá nepsaná pravidla, jako je inicializace a přehledný zápis:



Obr. 1.10: Inicializační krok.

1. Program začíná na inicializačním kroku, kde se začne vykonávat Action_019. Provádí zapsání log. 0 na VÝSTUP_1.

2. Abychom přešli do dalšího kroku, musíme splnit podmínku přechodu. Hodnota proměnné Spínač musí být logická 1. Uživatel ji musí manuálně sepnout. Pokud jsme sepnuli spínač a hodnota VÝSTUPU_1 je logická 0, přechod je platný a přejdeme do druhého kroku.
3. První krok již není aktivní a provádí se akce druhého kroku, která zapisuje na VÝSTUP_1 log. 1.
4. Jakmile je VÝSTUP_1 ekvivalentní log.1, tak bude následující přechod platný a program se dostane do Stop elementu, který program ukončí.

2 Jazyk SFC pro řadu LOGIX

V této kapitole budou postupně vysvětleny jednotlivé bloky, části a funkce jazyku SFC pro kontroléry řady LOGIX. Veškeré demonstrace jsou vytvořeny v programu Studio 5000 verze 33.12.

2.1 Struktura SFC_STEP

Každý vytvořený krok má svou strukturu, která obsahuje tagy určené pro různá použití - od detekování začátku nebo konce aktivity kroku až po sledování uběhlého času v daném kroku. Pro každý krok je k dispozici celkem 16 tagů. Rozsáhlá struktura tagů pro bloky, jako jsou kroky (steps) nebo akce, představuje jednu z významných výhod jazyka SFC. Uživatel má k dispozici předdefinované základní funkce, jejichž použití je velmi přehledné. Práce s těmito tagy je klíčová pro efektivní využívání a správnou implementaci SFC.

Name	Data Type	Description
Status	DINT	
X	BOOL	
FS	BOOL	
SA	BOOL	
LS	BOOL	
DN	BOOL	
OV	BOOL	
AlarmEn	BOOL	
AlarmLow	BOOL	
AlarmHigh	BOOL	
Reset	BOOL	
PauseTimer	BOOL	
PRE	DINT	
T	DINT	
TMax	DINT	
Count	DINT	
LimitLow	DINT	
LimitHigh	DINT	

Obr. 2.1: Struktura Stepu v Studiu 5000

2.1.1 Step.(X, FS, LS, SA)

Step.X je vždy aktivní pokud je aktivní jeho krok. Pomocí tohoto tagu můžeme předávat informaci o aktivitě daného kroku do jiných kroků, popřípadě jiných pod-

programů.

Step.FS neboli first scan je aktivní pouze při prvním skenu kroku. Můžeme ho využít pro krátký pulz na začátku spuštění kroku.

Step.LS neboli last scan je aktivní pouze při posledním skenu kroku. Obdobně jako u FS, můžeme získat krátký pulz, v tomto případě na konci kroku.

Step.SA bude aktivní vždy kromě prvního a posledního skenu. Můžeme si představit jako invertované FS+LS.[6]

2.1.2 Step.(T, PRE, TMAX)

Step.T se po spuštění kroku resetuje na nulovou hodnotu a začne měřit dobu, jak dlouho je krok aktivní v dané instanci. Jedno z jeho použití je využít ho jako časovač (viz kapitola 2.4).

Step.Pre je počáteční hodnota časovače, tedy doba, po kterou má časovač odpočívat, než dojde k aktivaci výstupu. Tato hodnota je nastavena uživatelem a určuje, jak dlouho má časovač čekat, než změří čas a aktivuje nebo deaktivuje příslušný výstup. Uživatel ji nemusí vyplňovat, je užitečná při tvoření časovače.

Step.TMAX určuje maximální dobu, po kterou byl krok aktivní během jakéhokoliv jeho aktivní doby. Používá se pro diagnostické účely.[6]

2.1.3 Step.(AlarmEn, LimitLow, LimitHigh)

Nastavením log.1 do **Step.AlarmEn**, povolíme použití alarmů Step. Limit společně s alarmy slouží jako ochrana proti brzkému ukončení (low) nebo pozdnímu zakončení kroku (high).[6]

LimitLow je hodnota času, která značí minimální dobu aktivního kroku. LimitHigh je jeho opakem, maximální čas aktivního kroku.

2.1.4 Step.(AlarmLow, AlarmHigh)

Pokud krok poruší limity, aktivuje se příslušný alarm, který je typu bool. Zůstane aktivní, dokud ji uživatel neresetuje.[6]

2.1.5 Step.(OV, Count, Status)

Step.OV slouží jako ověření správného času tagu Step.T, jestli hodnota času nepřešla do negativních hodnot.[6]

Step.Count je zabudovaný čítač do kroku. Inkrementuje o jedno při deaktivaci kroku a následné aktivaci kroku. Tento tag značně zjednodušil demonstrační úlohu

v praktické části práce. **Step.Status** je vhodný tag pro monitorování celého kroku. Je typu DINT a jednotlivé bity obsahují informaci o ostatních tagech kroku.

Tab. 2.1: Popis základních tagů kroku v SFC

Tag	Datový typ	Popis
X	BOOL	Aktivní po celou dobu trvání kroku
FS	BOOL	Aktivní pouze při prvním skenu
LS	BOOL	Aktivní pouze při posledním skenu
SA	BOOL	Aktivní kromě prvního a posledního skenu
T	DINT	Uplynulý čas v kroku
PRE	DINT	Počáteční hodnota časovače
DN	BOOL	Výstup časovače
LimitLow	DINT	Krok musí být aktivní po tuto dobu
LimitHigh	DINT	Krok nesmí být aktivní déle než tato hodnota
AlarmEn	BOOL	Povolení alarmů
AlarmLow	BOOL	Bude mít log.1, pokud se poruší LimitLow
AlarmHigh	BOOL	Bude mít log.1, pokud se poruší LimitHigh
TMax	DINT	Maximální doba, kterou krok dosáhl v jakékoliv aktivní době
OV	BOOL	Slouží pro diagnostiku a kontrolu Step.T
Count	DINT	Čítač průchodů krokem
Reset	BOOL	Resetování kroku
Status	DINT	Všechny tagy zapsané do jednoho

2.2 Struktura SFC_ACTION

Akce je nezbytnou součástí programu. V akcích můžeme vykonávat příkazy. V akci se používá jazyk ST. Její struktura, jak je znázorněno na následujícím obrázku, se skládá ze sedmi tagů, které umožňují efektivnější práci s akcí. Stejnojmenné tagy se strukturou SFC_STEP pracují na stejném principu.

Name	Data Type	Description
Status	DINT	
A	BOOL	
Q	BOOL	
PauseTimer	BOOL	
PRE	DINT	
T	DINT	
Count	DINT	

Obr. 2.2: Struktura Akce v Studiu 5000

2.2.1 Action.(Q, A)

Tag **Action.Q** je aktivní po celou dobu, s výjimkou posledního skenu programu. Tuto vlastnost lze využít například k předběžnému vypnutí specifického prvku před přechodem do dalšího kroku, čímž se zajistí plynulejší přechod mezi stavy.

Action.A je aktivní po celou dobu a lze jej využít k monitorování správného průběhu akce nebo k ovládání akčních prvků.[6]

2.2.2 Action.(T, PRE, PauseTimer)

Umožňují práci s uběhlým časem v akci pomocí .T nebo můžeme akci limitovat nastavením do tagu .PRE maximální hodnotu při jejímž dosažení se akce deaktivuje. Tag PauseTimer je aktivní pokud je SFC rutina pozastavená.[6]

2.2.3 Action.(Count, Status)

Action.Count je zabudovaný čítač do kroku. Inkrementuje o jedno při deaktivaci kroku a následné aktivaci kroku.

Action.Status zobrazuje veškeré tagy v jedné proměnné.

2.2.4 Boolean a Non-Boolean akce

Akce dosud použité v předchozích kapitolách byli vždy typu Non-Boolean, ale můžeme mít i akci typu Boolean. Je obdobou klasické akce, do které nemůžeme zapsat žádný kód, takže není schopna sama o sobě vykonávat příkazy. Musí se použít v kombinaci s jiným prvkem. A to tak, že můžeme tuto akci aktivovat v programu a její tag `.A` nebo `.Q` mít jako podmínku v jiné části. Narozdíl od Non-boolean akce, lze identickou akci použít několikrát v jednom SFC. [6]

Tab. 2.2: Popis základních tagů pro akce

Tag	Datový typ	Popis
Q	BOOL	Aktivní kromě posledního skenu v NON-Boolean akci
A	BOOL	Aktivní po celou dobu trvání akce
T	DINT	Uplynulý čas v akci
PRE	DINT	Hodnota časovače
Count	DINT	Čítač průchodů akce
Status	DINT	Zobrazuje veškeré tagy v jedné proměnné.

2.2.5 Posloupnost vykonání akcí v kroku

Akce v jednotlivých krocích se vykonávají postupně shora dolů, přičemž výsledný stav proměnných závisí na použitých kvalifikátorech. Pokud více akcí mění hodnotu stejné proměnné, tak výsledná hodnota proměnné odpovídá poslední provedené akci v pořadí.

Pokud akce obsahuje kvalifikátor S, SD nebo DS, její efekt může přetrvávat i v následujících krocích, a to až do doby, kdy je explicitně resetována pomocí kvalifikátoru R. Pro tyto kvalifikátory platí stejný způsob vyhodnocování jako u běžných akcí. Pokud v některém z následujících kroků existuje akce, která zapisuje do stejné proměnné, bude tato hodnota přepsána, protože akce se vyhodnocují postupně. Podrobnější popis těchto kvalifikátorů je uveden v kapitole 2.3.

2.3 Kvalifikátory akce

Kvalifikátor (anglicky "Qualifier") určuje, kdy a jak se má daná akce spustit, provádět nebo ukládat. Kvalifikátor je vždy umístěn v levém horním rohu každé akce a je volen

uživatel. Existuje celkem 11 kvalifikátorů, které jsou klíčové pro práci s jazykem SFC.

2.3.1 N: Non-stored

Kvalifikátor N je základní kvalifikátor. Akce, která má kvalifikátor N, se bude vykonávat pokud je její přidružený krok aktivní. (mateřský krok, odvozený krok, vlastní krok).

2.3.2 S: Stored

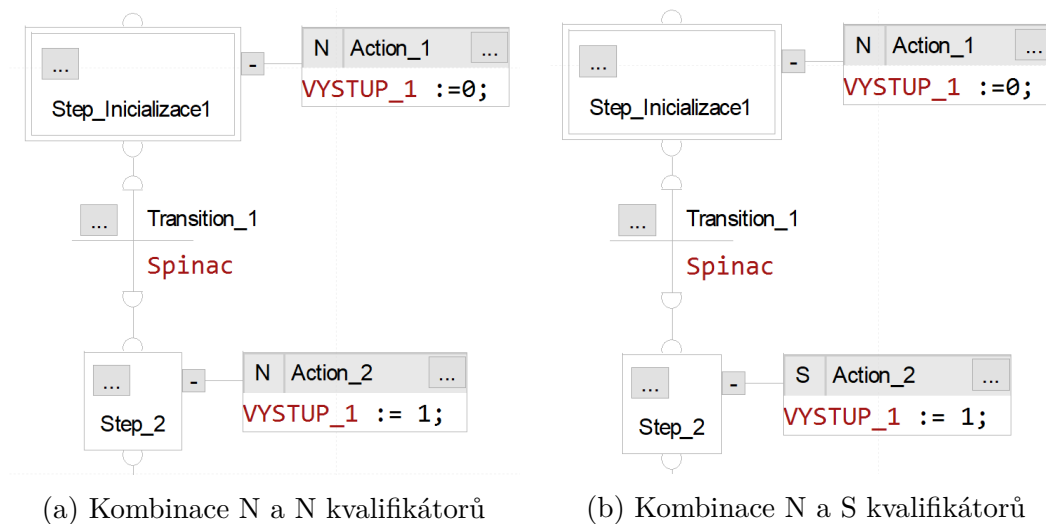
Akce, která má kvalifikátor S, se bude vykonávat v každém cyklu a to i ve chvíli kdy její krok není aktivní. Pokud ji chceme deaktivovat, musíme tak udělat pomocí kvalifikátoru R v jiné akci.

2.3.3 R: Reset

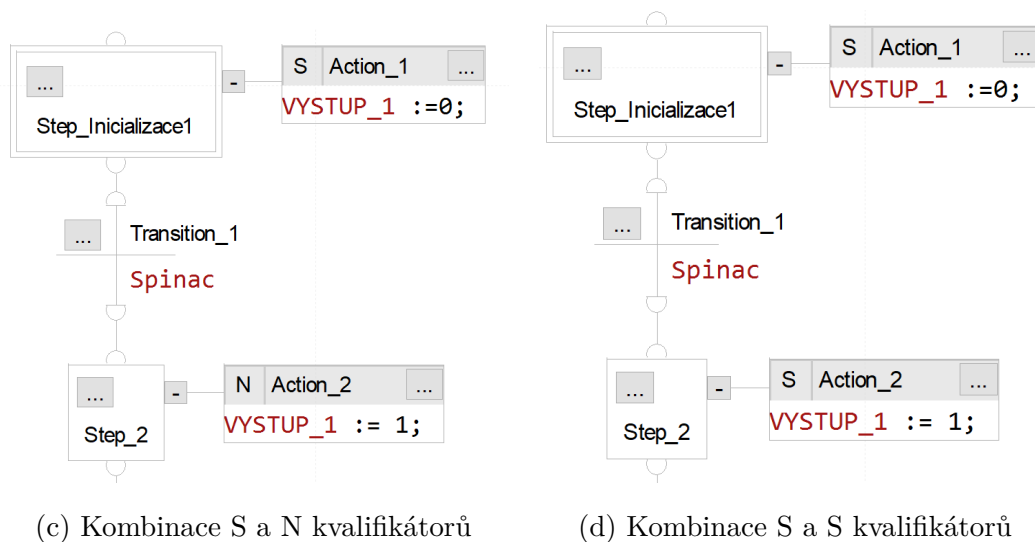
Slouží pro reset akce s kvalifikátorem S. Do akce s kvalifikátorem R nelze zapsat žádný kód. Jsme schopni pouze vybrat akci, kterou potřebujeme resetovat.

2.3.4 Porovnání N, S, R

Na následujícím obrázku je několik příkladů pomocí kterých zapisujeme na výstup log. 1. Vždy se mění jeden z kvalifikátorů dvou akcí.



Obr. 2.3: Porovnání N s S kvalifikátorem



Obr. 2.4: Porovnání S s N kvalifikátorem

a) Kombinace N a N

Po spuštění programu se začne vykonávat první akce. Zapiše na VYSTUP_1 logickou 0.

Čekáme na sepnutí spínače a a po jeho sepnutí se dostaneme do druhého kroku. Zde se stane druhá akce aktivní a zapiše na výstup logickou 1.

b) Kombinace N a S

Po spuštění programu se začne vykonávat první akce. Zapiše na výstup naší proměnné logickou 0.

Čekáme na sepnutí spínače a a po jeho sepnutí se dostaneme do druhého kroku. Zde se stane druhá akce aktivní a zapiše na výstup logickou 1. Tato akce je ale nyní uložena v paměti a bude se vykonávat pořád dokud ji neresetujeme. V tomto případě nenastane žádný viditelný rozdíl mezi případem a).

c) Kombinace S a N

Po spuštění programu se začne vykonávat první akce. Zapiše na výstup naší proměnné logickou 0 a akce se uloží do paměti.

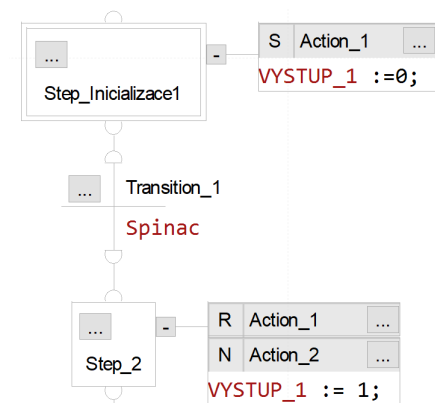
Čekáme na sepnutí spínače a a po jeho sepnutí se dostaneme do druhého kroku. Zde se stane druhá akce aktivní a zapiše na výstup logickou 1. Dostaneme stejný výsledek jako u předchozích, ale pokud bychom měli v tomto kódu třetí krok ve kterém bychom pracovali s jiným výstupem, tak v momentě kdy program bude ve třetím kroku, Výstup_1 se nastaví zpět do logické 0.

d) Kombinace N a S

Po spuštění programu se začne vykonávat první akce. Zapiše na výstup naší proměnné logickou 0 a akce se uloží do paměti.

Čekáme na sepnutí spínače a a po jeho sepnutí se dostaneme do druhého kroku. Zde se stane druhá akce aktivní a protože paměť akcí je typu LIFO, tak i přes aktivní Action_1 se zapiše na výstup logická 1, Action_2 má přednost.

Abychom dokázali správně využít kvalifikátor S, musíme ho resetovat pomocí kvalifikátoru R. Na stejném příkladě bychom mohli použít reset podle obrázku 2.5.



Obr. 2.5: Resetování akce s kvalifikátorem S

2.3.5 Kvalifikátory P, P0, P1

Tyto kvalifikátory říkají akci, aby byla aktivní na specifický pulz.

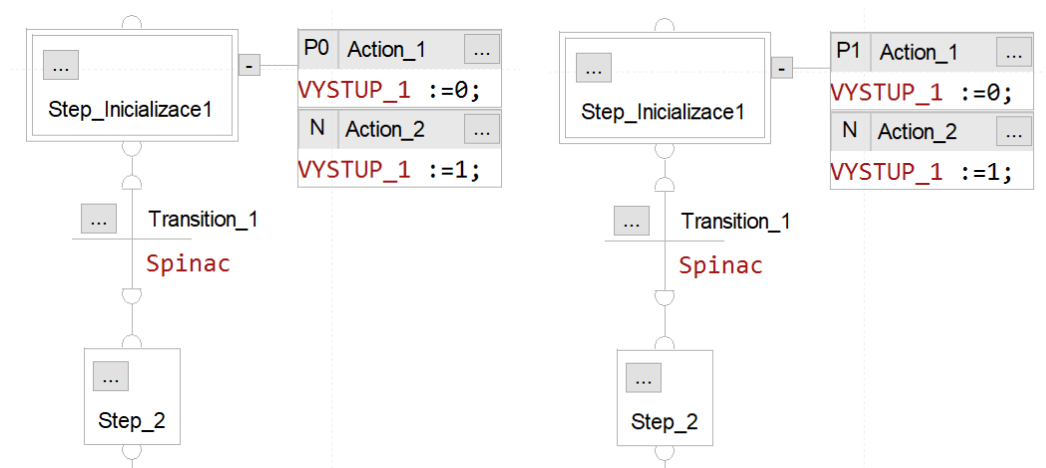
P: Impuls znamená, že akce bude aktivní při nástupné i sestupné hraně kroku. To znamená, že se akce aktivuje dvakrát za jeden krok, při prvním a posledním skenu.

P0: Impuls (Sestupná hrana) znamená, že akce bude aktivní jen při sestupné hraně řídicího kroku.

P1: Impuls (Nástupná hrana) znamená, že akce bude aktivní pouze při nástupné hraně mateřského kroku. Kombinací P1 a P0 lze efektivně měnit proměnnou v jednom kroku.

2.3.6 Porovnání P, P0, P1

Na obrázku 2.6 jsou příklady které ukazují, jak funguje P0 a P1 společně s kvalifikátorem N. Na obrázku 2.7 je příklad s kvalifikátorem P.



(a) Kvalifikátor P0

(b) Kvalifikátor P1

Obr. 2.6: Porovnání P0 a P1

a) Kombinace P0 a N

Po spuštění programu se začne vykonávat Action_2. Vykonává se po celou dobu, co jsme v prvním kroku. V momentě, kdy splníme podmínku přechodu, tak se krok deaktivuje a jeho sestupná hrana aktivuje naši akci Action_1.

Ve výsledku to znamená, že při startu programu bude na výstupu log. 1 a jakmile stiskneme spínač a přesuneme se do dalšího kroku, na výstup se zapíše log.0.

Mohli bychom zde mít místo P0 kvalifikátor P a výsledek by byl stejný. Jediný rozdíl by byl krátký pulz na začátku, který by zapsal na výstup log. 0.

b) Kombinace P1 a N

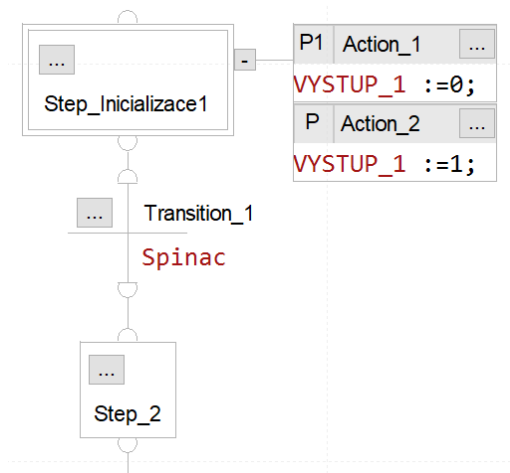
Po spuštění programu se vykoná Action_1 a zapíše na výstup log 0, ale hned v dalším skenu se hodnota přepíše akcí Action_2 na logickou 1.

V druhém kroku bude na výstupu logická 1.

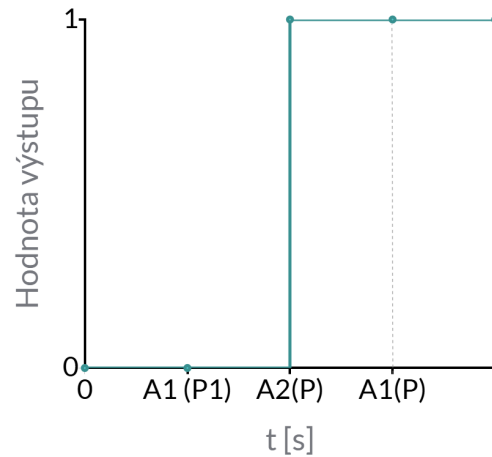
a) Kombinace P1 a P

Na začátku programu se nám obě akce vykonají, ale protože se akce vykonávají v každém kroku postupně od zhora dolů, nejdříve se provede Action_1 a poté Action_2. Na konci se provede ještě jednou Action_2.

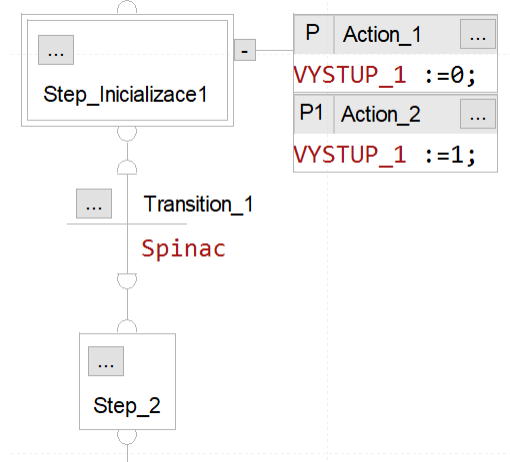
To znamená, že na začátku se zapíše log.0, poté se ihned přepíše na log.1. a po sepnutí spínače se dostaneme do druhého kroku a na výstupu bude log.1.



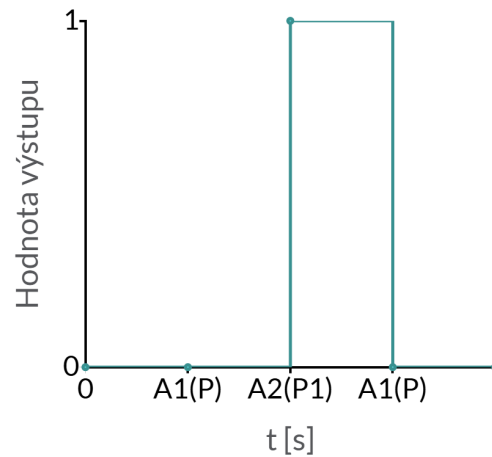
(a) Qualifier P1 a P



(a) Výstup programu P1_P



(b) Qualifier P a P1



(b) Výstup programu P_P1

Obr. 2.7: Kvalifikátor P

b) Kombinace P a P1

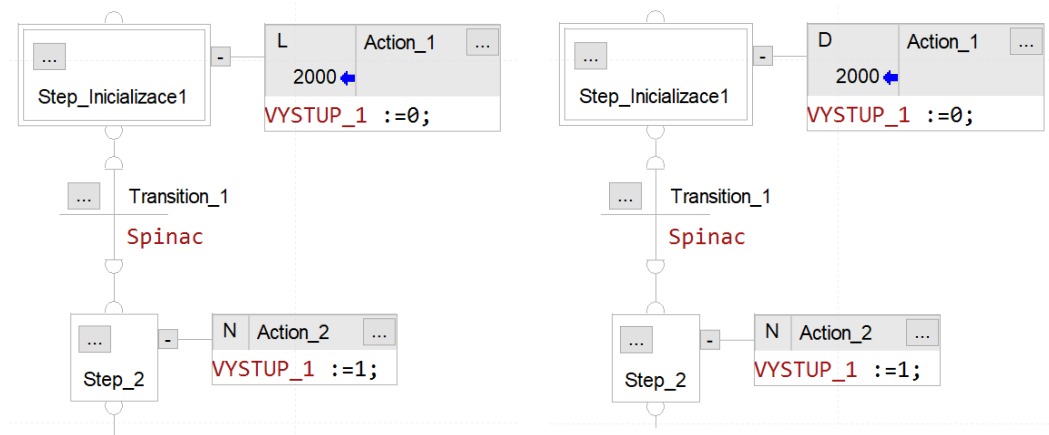
Stejně jako u předchozího příkladu se nejdříve provede Action_1 a poté Action_2. Během toho, co bude inicializační krok aktivní, bude na výstupu log.1. Rozdílem je, že po splnění podmínky přechodu se provede znovu Action_1 a zapíše na výstup log.0. Takže v druhém kroku bude na výstupu logická 0.

2.3.7 Kvalifikátory L, D

Tyto kvalifikátory fungují podobně jako N, rozdílem je, že zároveň akci časově limitují L a nebo časově opozdí D.

L: Časové omezení znamená, že akce se spustí v momentu aktivace řídicího kroku, ale ve vlastnostech akce můžeme nastavit maximální čas, kdy se má akce provádět. Po uplynutí nastaveného času se akce vypne i přesto, že krok je stále aktivní.

D: Časově opožděná znamená, že po aktivaci řídicího kroku se akce spustí opožděně o námi nastavený čas. Po opožděném spuštění bude aktivní dále společně s řídicím krokem.



(a) Qualifier L

(b) Qualifier D

Obr. 2.8: Porovnání L a D

a) Kombinace L a N

Po startu programu se nastaví na výstup logická hodnota 0. Po 2 sekundách se akce Action_1 deaktivuje. Přejít do dalšího kroku může nastat poté co se tak stane nebo dříve a po přechodu se zapíše na výstup logická hodnota 1

b) Kombinace D a N

Po startu programu se v prvním kroku nic neděje. Po 2 sekundách se akce Action_1 aktivuje a nastaví na výstup logickou hodnota 0.

Přejít do dalšího kroku může nastat dříve než se Action_1 provede a nemusí se za celý program aktivovat ani jednou.

2.3.8 Kvalifikátory SD, SL, DS

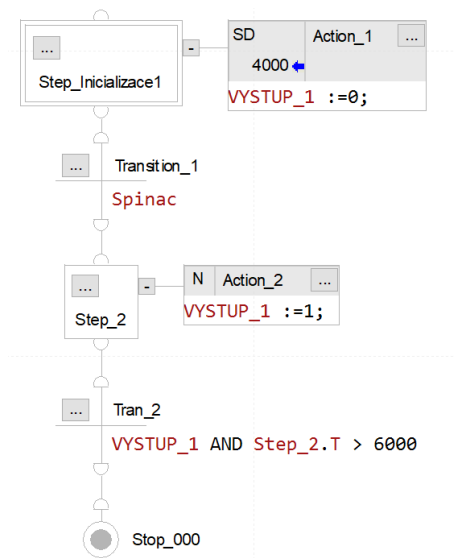
Jsou kombinací ukládání akcí pomocí kvalifikátoru S s opožděním D nebo časově omezené L. Tyto akce se mohou resetovat pomocí kvalifikátoru R stejně jako S.

SD: Uložená a časově opožděná znamená, že se akce bude vykonávat při každém cyklu, ale spustí se až po uplynutí nastaveného času.

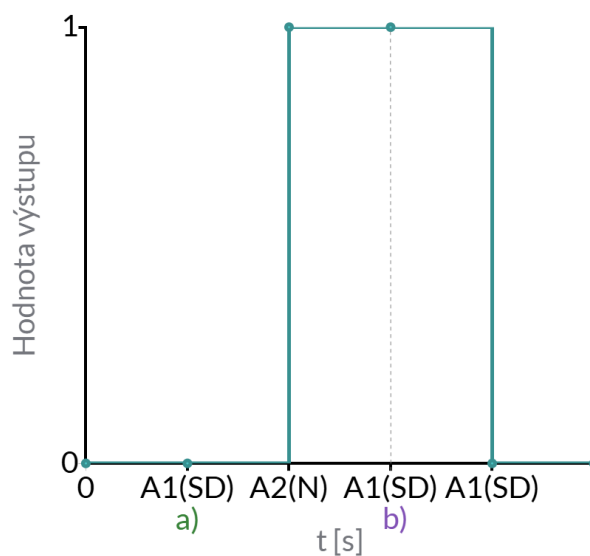
SL: Uložená a časově omezená znamená, že se akce bude vykonávat v každém cyklu, ale po uplynutí nastaveného času se akce deaktivuje.

DS: Časově opožděná a uložená Akce nejdříve musí v řídicím kroku dosáhnout nastaveného času, aby se mohla aktivovat a poté se bude vykonávat v každém cyklu.

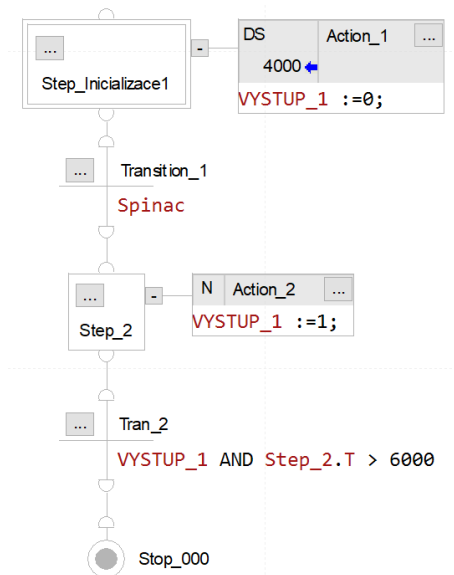
2.3.9 Rozdíl mezi SD a DS



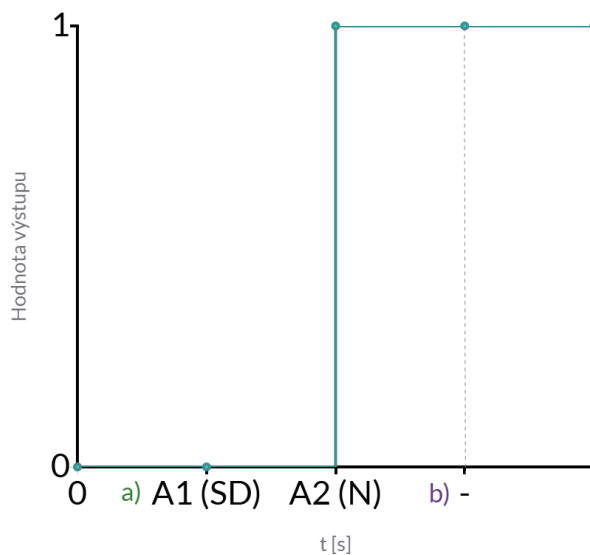
(a) Qualifier SD



(a) Výstup programu SD_N



(b) Qualifier DS



(b) Výstup programu P_P1

Obr. 2.9: Porovnání SD a DS

a) Kombinace SD a N

Program spustíme, Action_1 se uloží do paměti a nyní máme 2 možnosti. Můžeme počkat, než se akce spustí a zapíše na výstup log.0 (Případ a) v trendu programu), nebo můžeme ihned splnit podmínku přechodu a postoupit dále (Případ b) v trendu programu). V tomto případě toto nebude mít vliv na výsledek.

Pokud počkáme, a poté přejdeme do druhého kroku, tak se v druhém kroku zapíše na výstup logická 1 a nyní se čeká na splnění podmínky, pokud budeme v druhém kroku déle jak 6 s, program přejde dále, a uložená akce po ukončení zapíše na výstup logickou 0. Kdybychom se ale rozhodli nečekat, tak by výsledek byl stejný.

b) Kombinace DS a N

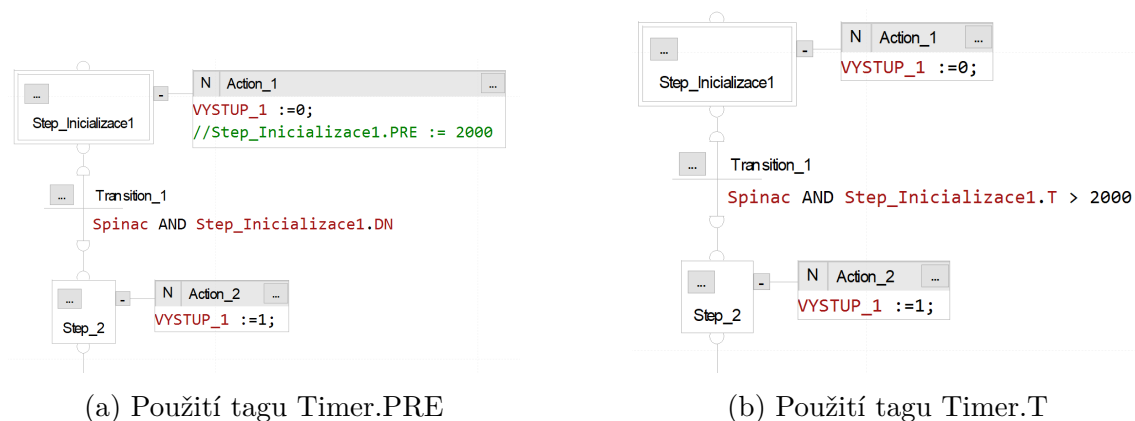
Program je velmi podobný, ale jelikož se Action_1 ukládá až po uplynutí 4 sekund, je možné ji přeskočit a nestihla by se uložit. To znamená, že na konci nám zůstane na výstupu logická 1. V trendu programu je možnost a) - nechali jsme akci uložit a za b) přeskočili jsme první akci a nestihla se uložit.

Tab. 2.3: Popis kvalifikátorů pro akce

Tag	Popis
N	Aktivní po dobu kroku
S	Aktivní dokud není resetována
R	Resetuje uloženou akci
P	Aktivní pulzově na začátku a na konci
P0	Aktivní pulz pouze na konci
P1	Aktivní pulz pouze na začátku
L	Aktivní dokud nevyprší nastavený čas v kroku
D	Opožděně aktivní v daném kroku
SD	Nejprve se uloží a poté se stane opožděně aktivní
SL	Nejprve se uloží a poté je aktivní do vypršení času
DS	Nejprve je opožděná a po vypršení času se uloží

2.4 Použití časovače v jazyku SFC

Nejjednodušší způsob, jak vytvořit časovač, je využití tagů struktury kroku. Každý krok obsahuje tagy Step.T, Step.PRE a Step.DN, jejichž podrobný popis naleznete v kapitole 2.1.2). Na obrázku 2.10 je znázorněno, jak lze tyto tagy použít k vytvoření časovače.



Obr. 2.10: Využití interního časovače struktury SFC_STEP

a) Časovač pomocí .PRE a .DN

Ve vlastnostech kroku je nastavená hodnota .PRE na 2 sekundy. Můžeme ji nastavit i přímo v programu (komentář v první akci). Po uplynutí nastaveného času se zapíše do .DN logická 1, což splní podmínku našeho přechodu.

b) Časovač pomocí .T

Lze také použít pouze uplynulý čas .T pro vytvoření podmínky. Podmínka přechodu musí poté porovnávat právě tento čas.

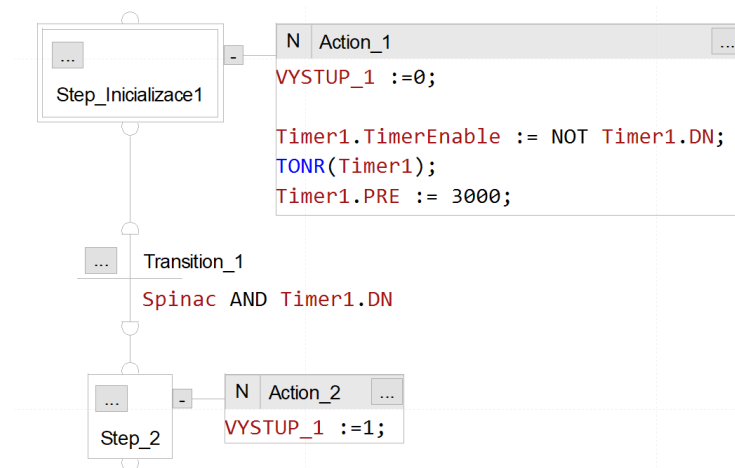
Je také možné použít funkci FBD časovače a pracovat s ním pouze v akci (viz v obrázek 2.11).

V demonstrační úloze byla převážně použita metoda pomocí .T tagu.

2.5 Subrutiny

Při práci se subrutinami v jazyce SFC se využívají tři hlavní instrukce: JSR, SFR a blok SBR/RET. Je možné vytvořit až 25 vnořených subrutin a pomocí bloku SBR/RET předávat a vracet až 40 parametrů.

Funkce SFR umožňuje resetování specifické rutiny na libovolný krok a může být využita pro návrat ze subrutiny.[6]



Obr. 2.11: Použití FBD funkce časovače v akci pomocí jazyku ST

2.6 Ukončení SFC rutiny

Program můžeme ukončit několika způsoby. Pomocí Stop elementu nebo pokud chceme aby program po jeho dokončení se opakoval, můžeme použít na konci příkaz SFR.

Stop element se po jeho aktivaci musí resetovat nastavením bitu Stop.X na hodnotu log.0. Místo toho lze na konec programu přidat akci s funkcí SFR, která umožní návrat na specifikované místo v programu, nebo lze na konec vložit prázdný krok, který nebude možné resetovat příznakem jako stop element.

V demonstrační úloze byla nejčastěji použita funkce SFR, protože je možné zvolit místo do kterého se SFC má vrátit.[6]

2.7 Dokumentace SFC: Nastavení tisku rutin

Každá SFC rutina je zobrazena ve formě mřížky složené ze dvou sloupců a libovolného počtu řádků. Pokud se veškerý kód nachází uvnitř jednoho čtverce této mřížky, je rutina při použití funkce „Print Routine“ vytištěna správně. V případě, že kód přesahuje hranice jednoho čtverce, může dojít při tisku ke zkreslení zobrazení rutiny.

V nabídce Print Options → SFC X lze nastavit tři možnosti pro tisk:

- Shrink to Fit Page: Deformuje SFC, tak aby byl na jedné stránce
- Include Blank Sheets: Bude obsahovat prázdné mřížky
- Include SFC Element Properties Listings: Bude obsahovat vlastnosti SFC

Pokud kód rutiny přesahuje hranice jednotlivých čtverců mřížky, je vhodné tyto volby deaktivovat a při tisku ručně zvolit konkrétní části mřížky, které obsahují kód.

3 Demonstrační úloha

Tato část práce se zaměřuje na konkrétní úlohu, ve které budou demonstrovány funkce jazyka SFC, jeho výhody a situace, ve kterých SFC není vhodné použít. Úloha je naprogramována v prostředí Studio 5000 pro fyzický laboratorní model (viz příloha XXX) . K úloze byla vytvořena vizualizace v programu FactoryTalk View Studio ME. Vizualizace je vytvořena tak, aby uživatel mohl sledovat jednotlivé kroky SFC programu.

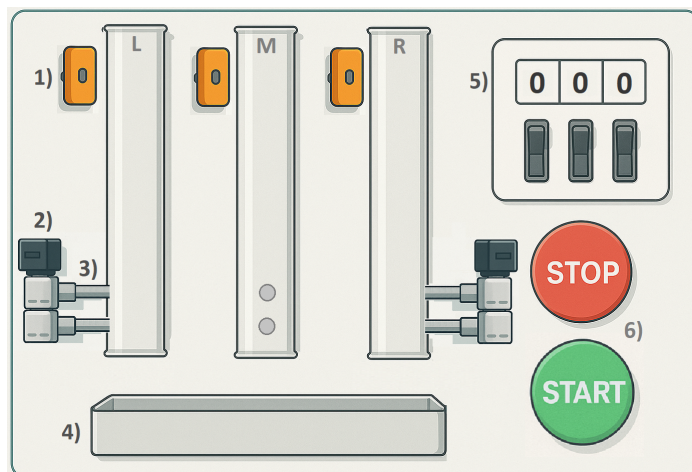
3.1 Popis demonstrační úlohy: Kuličky

Demonstrační úlohou je laboratorní modul s názvem Kuličky. Tato úloha je jeden z projektů v předmětu BPC-PGA a tato práce se drží jeho zadání, které bude v této kapitole popsáno.

Model dává kuličky ve třech samostatných válcích na základě vstupního čísla zadaného pro každý válec. Číslo pro každý válec můžeme zadat pomocí otočného číslicového spínače (rozsah každé cifry je 0 až 9).

Válce jsou vybaveny 2 západkami, které umožňují postupně provádět dávkování. Západky pro prostřední válec jsou umístěny ze zadní strany. Pro chlazení západek jsou k nim připojeny ventilátory. Válce provádějí dávkování současně a jsou na sobě nezávislé.

Zda je válec naplněn, určují polohové snímače umístěné v horní části válců. Pod válci se nachází krabice na kuličky. Její detekce se provádí pomocí mikrospínače. Tlačítka START a STOP se používají pro ovládání modulu a zároveň jako indikátory. Mají v sobě zabudované červené světlo - STOP a zelené světlo - START.



Obr. 3.1: Vizuální reprezentace laboratorního modelu: Kuličky [Příloha A]

Každý válec má vlastní ventilátor, snímač a západky. Prostřední válec má nainstalované západky s ventilátorem ze zadní strany modulu. V programu jsou označovány válce jako L, M, R neboli left, middle, right. Na obrázku 3.1 jsou čísla odpovídající legendě:

- 1) - Polohový snímač pro detekci plného válce
- 2) - Ventilátor pro chlazení západek
- 3) - Horní a spodní západky pro dávkování
- 4) - Krabíčka do které padají kuličky
- 5) - Otočný číselný spínač
- 6) - Tlačítka START a STOP pro ovládání modulu a indikaci

Dle zadání úlohy jsou vstupní a výstupní proměnné modulu přiřazeny do PLC a zapsány do tabulky 3.1.

Tab. 3.1: Vstupy a výstupy modulu a jejich zápis do PLC tagů

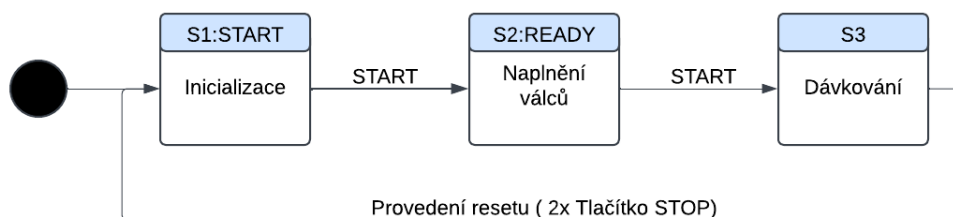
Název	Typ	Popis
BCD_L_0	DI 0	BCD levý válec (2^0)
BCD_L_1	DI 1	BCD levý válec (2^1)
BCD_L_2	DI 2	BCD levý válec (2^2)
BCD_L_3	DI 3	BCD levý válec (2^3)
BCD_M_0	DI 4	BCD střední válec (2^0)
BCD_M_1	DI 5	BCD střední válec (2^1)
BCD_M_2	DI 6	BCD střední válec (2^2)
BCD_M_3	DI 7	BCD střední válec (2^3)
BCD_R_0	DI 8	BCD pravý válec (2^0)
BCD_R_1	DI 9	BCD pravý válec (2^1)
BCD_R_2	DI 10	BCD pravý válec (2^2)
BCD_R_3	DI 11	BCD pravý válec (2^3)
S1	DI 16	Čidlo naplnění kuliček v levém válci
S2	DI 22	Čidlo naplnění kuliček ve středním válci
S3	DI 18	Čidlo naplnění kuliček v pravém válci
S4	DI 21	Čidlo přítomnosti krabice
ZH1	DI	Zpětné hlášení od ventilátoru západek v levém válci
ZH2	DI	Zpětné hlášení od ventilátoru západek ve středním válci
ZH3	DI	Zpětné hlášení od ventilátoru západek v pravém válci
STOP	DI 20	Červené tlačítko STOP
START	DI 23	Zelené tlačítko START
No_Balls1	Číslo	Počet kuliček, kolik se má odpočítat v levém válci
No_Balls2	Číslo	Počet kuliček, kolik se má odpočítat ve středním válci
No_Balls3	Číslo	Počet kuliček, kolik se má odpočítat v pravém válci
AlarmLX	-	Signalizace, že No_BallsX jsou menší než 0
AlarmHX	-	Signalizace, že No_BallsX jsou větší než 9
C_LED	DO 11	Červená LED
Z_LED	DO 9	Zelená LED
M1U	DO 13	Zarážka levá horní
M1D	DO 1	Zarážka levá dolní
M2U	DO 2	Zarážka střední horní
M2D	DO 3	Zarážka střední dolní
M3U	DO 6	Zarážka pravá horní
M3D	DO 7	Zarážka pravá dolní
Ve1	DO 0	Spouštění ventilátoru pro levé západky
Ve2	DO 8	Spouštění ventilátoru pro střední západky
Ve3	DO 8	Spouštění ventilátoru pro pravé západky

Popis funkce modulu

Modul se spouští stisknutím tlačítka Start, čímž se aktivují západky a uživatel může jednotlivé válce naplnit. Uživatel nastaví na číslicových spínačích požadovaný počet kuliček, které mají propadnout do krabice z každého válce. Každý válec musí být naplněn až po úroveň horního snímače tak, aby se všechny snímače aktivovaly.

Jakmile jsou všechny válce naplněny, model je připraven k dávkování, což je indikováno rozsvícením zeleného indikátoru START. Po opětovném stiknutí tlačítka START se kuličky začnou postupně dávkovat ze všech válců současně, což umožňují dvě programově ovládané západky, které se střídavě spínají a rozepínají. Po propadnutí všech požadovaných kuliček je možné válce znovu naplnit a celý proces opakovat.

Resetování programu probíhá pomocí tlačítka STOP. Sepnutím tlačítka se program pozastaví a rozsvítí se červený indikátor. Uživatel má dvě možnosti: stiknutím tlačítka START se program vrátí, kde skončil, a pokračuje. Stiknutím tlačítka STOP se program resetuje na začátek.[5]



Obr. 3.2: Postup uživatele při průchodu dávkování

Každý číslicový volič má na svém výstupu čtyři vodiče neboli čtyři bitové hodnoty, které dohromady tvoří číslo v BCD kódu. Výstupy z voliče jsou převedeny na binární signály a v programu se musí převést na celé číslo.

Požadavky na proces

Program lze zastavit v libovolném okamžiku tlačítkem STOP, kdy dojde k rozsvícení červeného světla – stav STOP. Opětovným stisknutím tlačítka STOP se program resetuje na první krok, uvolní se západky a zbylé kuličky se vysypou do krabice.

Pokud uživatel nechce program ukončit, může se vrátit na poslední krok před pozastavením tlačítkem START.

Při běhu programu budou ventilátory vždy sepnuty a bude se ověřovat běh ventilátorů pomocí zpětného hlášení (simulovaný čas sepnutí je 5 s) z příslušného stykače. Hodnotu číslicového voliče lze nastavovat v rozsahu 0 až 9. Aplikace signalizuje, že

zadaná hodnota je mimo rozsah ($\text{AlarmL} < 0$ a $\text{AlarmH} > 9$). V tomto případě nemůžete program spustit tlačítkem START (ve stavu START ani READY).[5]

3.2 Programování demonstrační úlohy

V této kapitole budou popsány jednotlivé části kódu, jejich vlastnosti a účel. Na úloze lze pozorovat, kde SFC je schopno ulehčit programátorovi práci a kde naopak ztížit. Pro porovnání je hlavní řídicí rutina naprogramována i v jazycích LAD a ST. Kód byl vytvořen ve aplikaci Studio 5000 pro LOGIX kontrolér 1769. Celý kód naleznete v elektronické příloze D.

3.2.1 Přehled a architektura programu

Program se vykonává pod jedním PLC taskem a má dohromady 6 programů s jejich příslušnými rutinami. Hlavní rutina se nazývá MainRoutineSFC a ovládá postup procesem.

1. MainTask

(a) MainProgram

- i. SFC_MainRoutine - ovládá průběh programu
- ii. DavkovaniL - dávkování pro levý válec
- iii. DavkovaniM - dávkování pro prostřední válec
- iv. DavkovaniR - dávkování pro pravý válec
- v. LAD_MainRoutine - hlavní rutina v jazyce LAD
- vi. ST_MainRoutine - hlavní rutina v jazyce ST

(b) StopProgram

- i. DetekceStop - čeká na sepnutí tlačítka STOP a spouští subrutinu StopRoutine
- ii. StopRoutine - program je pozastaven a čeká na sepnutí START nebo STOP

(c) BCD_PrevodProgram

- i. BCD_PrevodRoutine - převod výstupů číslicového spínače na celé číslo

(d) AlarmProgram

- i. AlarmsRoutine - detekuje hodnoty překračující meze číslicového spínače

(e) Informace_HMI_Logic

- i. Informace_SFC_LogicRoutine - logika pro informační okno v HMI

(f) TlacitkaSetupProgram

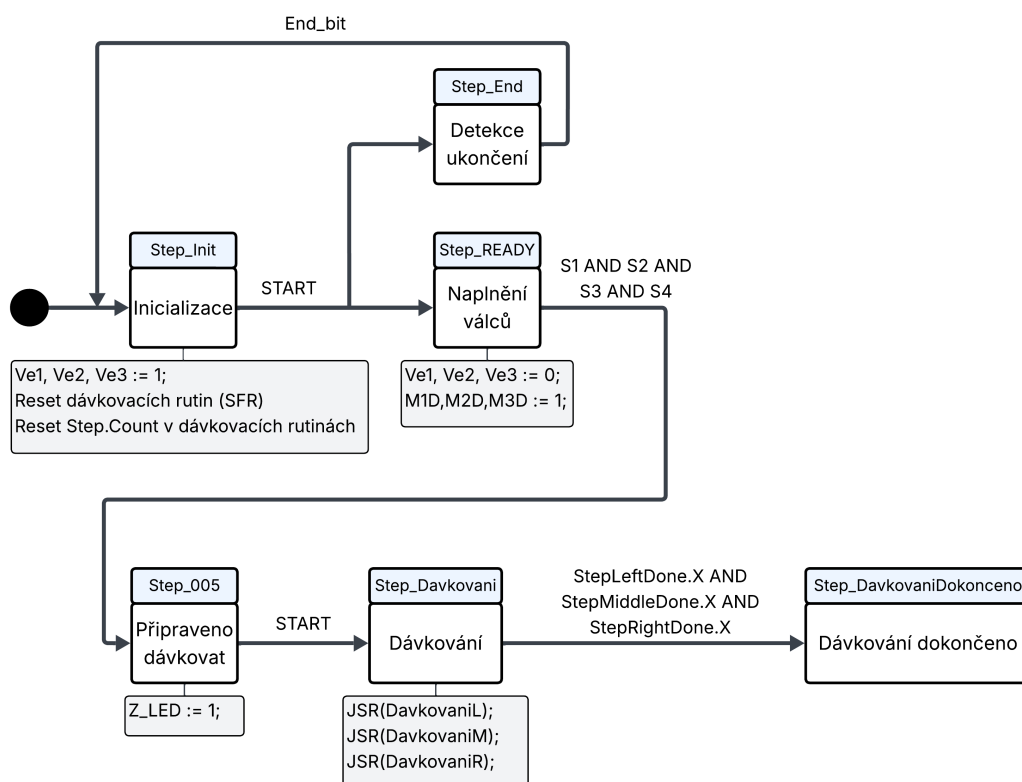
- i. StopStartTlacitkaRoutine - nastavuje tlačítka tak, aby reagovala na nástupnou hranu

3.2.2 MainProgram: SFC_MainRoutine

Rutina SFC_MainRoutine je hlavní řídicí prvek, který spouští dávkování a resetuje program. Prvním krokem je inicializační krok, který programově resetuje dávkovací rutiny a vypne ventilátory (dle zadání by měly mít zapnuty již od začátku, ale z praktického hlediska jsou v tomto kroku vypnuty).

Po stiknutí tlačítka START se program rozdělí do dvou paralelních větví. V této části bude popsána větev, která začíná krokem Step_Ready, ve které probíhá postupně program.

Druhá větev začíná krokem Step_DetectingEnd a stará se o resetování programu na iniciační krok (viz v obrázek 3.3). V tomto diagramu nejsou zobrazeny všechny



Obr. 3.3: Diagram hlavní rutiny

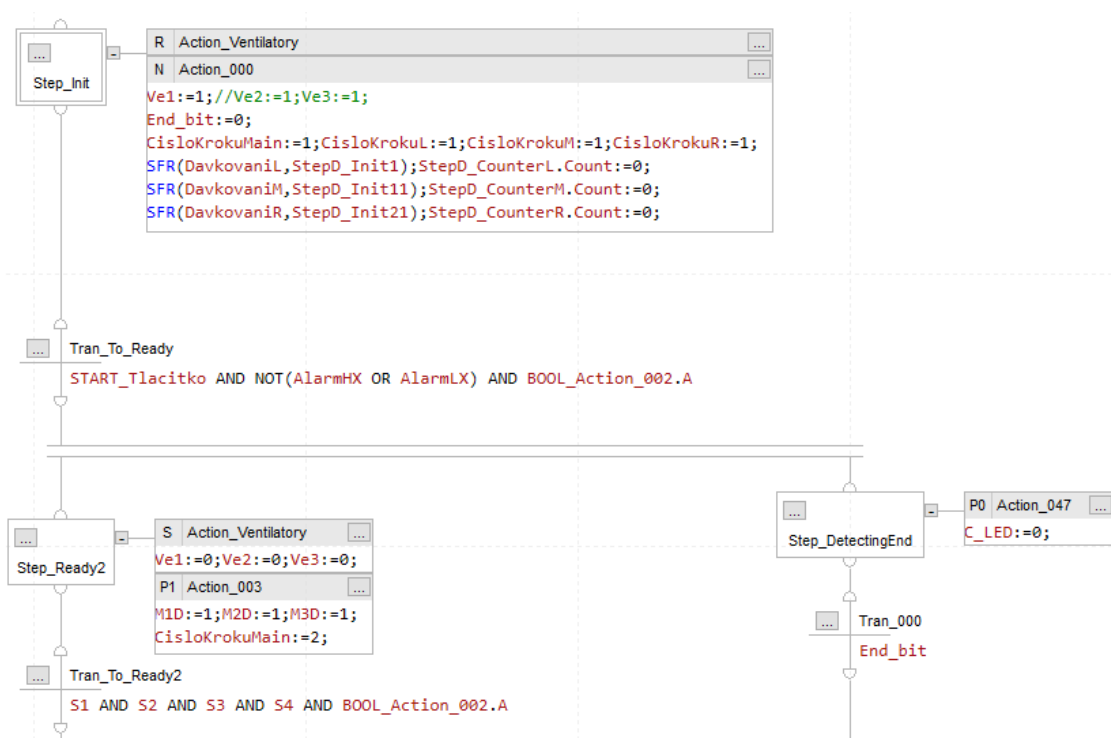
podmínky a úkony, které se v jednotlivých krocích provádějí. V následujících podkapitolách budou jednotlivé části rozebrány do detailů.

StepInit a StepReady

V těchto krocích nejsou žádné komplikace pro tento jazyk. V prvním kroku pomocí příkazu SFR jsou resetovány rutiny dávkování. Zároveň zapisují 0 do tagů Step.Count, a to konkrétně pro kroky v rutinách dávkování, které pomocí .Count počítají kuličky (viz kapitola 3.2.3).

V přechodu na druhý krok je zajištěné pomocí **AlarmHX** a **AlarmLX**, že číslo požadovaných kuliček nepřesahuje meze. Zároveň je zde tag booleanské akce **BOOL_Action_002.A**. Tato akce je aktivní pouze pokud nejsme ve stavu STOP (Program pozastaven). Je umístěna v rutině DetekceStop a je použita i v dalších podmínkách.

V druhém kroku jsou spuštěny ventilátory v akci s kvalifikátorem S, takže akce



Obr. 3.4: Step_Init a Step_Ready

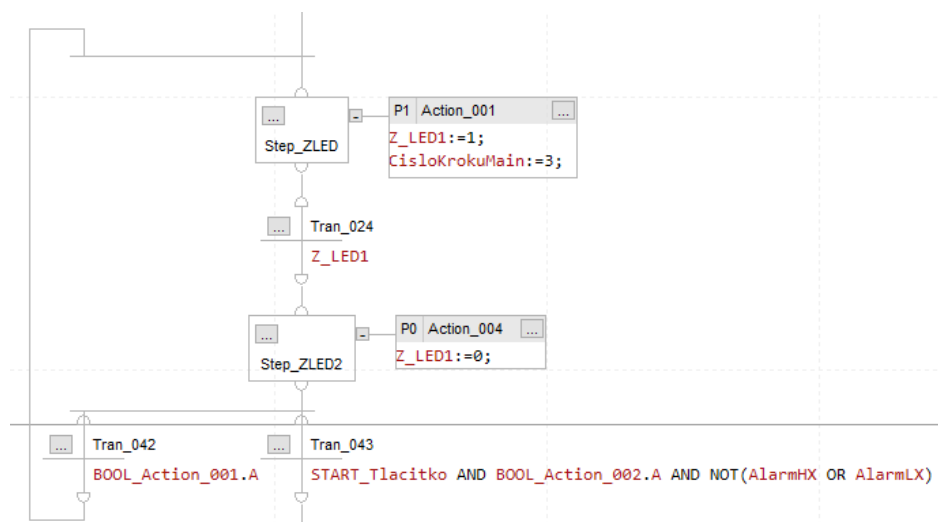
bude probíhat, dokud není resetována v prvním kroku akcí s kvalifikátorem R. Uživatel v tuto chvíli může plnit válce, protože byly zapnuty spodní západky. Po splnění podmínek se pokračuje na třetí krok.

Step_ZLED a Step_ZLED2

Tyto dva kroky reprezentují jeden krok na obrázku 3.3. V kroku se rozsvítí zelený indikátor jako potvrzení, že je možné začít dávkování stisknutím tlačítka START.

V této části kódu se ukazuje, že paralelní procesy v SFC nejsou vhodné. Pokud by byly tyto dva kroky pouze v jednom, tak při pozastavení programu tlačítkem STOP a následným sepnutím tlačítka START, program je v jednom skenu schopen přeskočit až na další krok Step_Davkovani.

Aby tento problém nenastával, je vytvořena výběrová větev do dvou podmínek a smyčka zpět, takže krok je rozdělen do dvou. Toto řešení není ideální a v dalších krocích je situace podobná a vyřešena jiným způsobem.



Obr. 3.5: Step_ZLED

Step_Davkovani a Step_DavkovaniDokonceno

Po příchodu do tohoto krok se pomocí JSR skočí do dávkovacích subrutin. Proměnná Davkovani_InProgress bude mít hodnotu logické 1 po dobu kroku a jakmile z kroku budeme přecházet na další, tak na posledním skenu se nastaví na hodnotu logické 0. Je zde podobný problém jako u předchozího kroku Step_ZLED a to při pozastavení rutiny se musí zajistit, aby dávkování bylo pozastaveno a to znamená, že akce s příkazem JSR nemůže být aktivní. To by se dalo řešit smyčkou zpět jako v kroku Step_005 nebo se dá tento problém obejít pomocí podmínky IF, jak je vyřešeno kódu.

Toto řešení je elegantnější, ale vlastně takové řešení obchází celý princip sekvenčního programování.



Obr. 3.6: Step_Davkovani a Step_DavkovaniComplete

Do posledního kroku se dostaneme, pokud budou aktivní kroky v jednotlivých dávkovacích rutinách. To je vyřešeno pomocí tagu těchto kroků Step.X. Zároveň je zde podmínka z HMI PotvrditDavkovani, která je pouze jako tlačítko na HMI (viz kapitola 3.3)

Jakmile se dostaneme do posledního kroku, pomocí tlačítka STOP se program resetuje na inicializační krok.

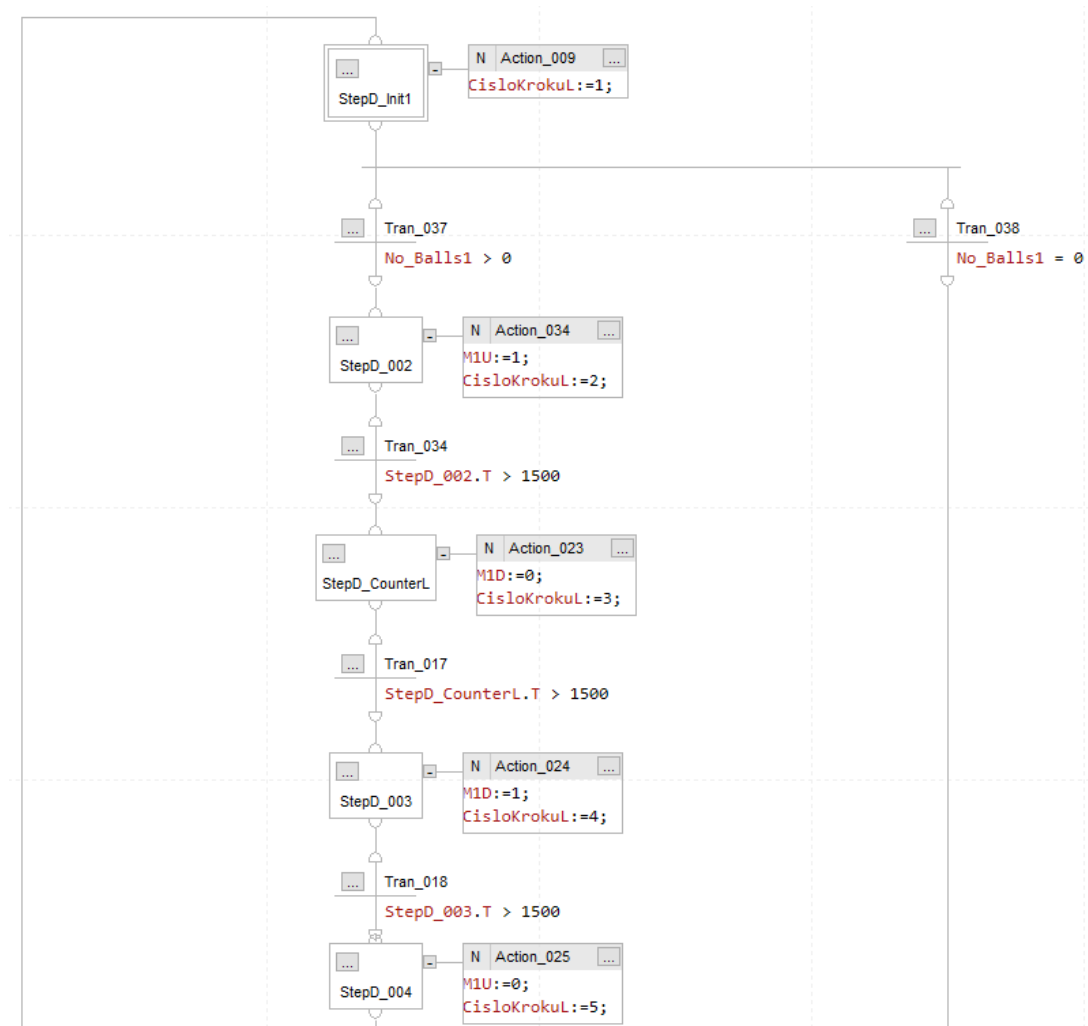
3.2.3 MainProgram: DávkováníL,DávkováníM a DávkováníR

Vzhledem k tomu, že pro jazyk SFC nelze použít ADD-ON instrukce, jsou vytvořeny tři téměř identické rutiny pro každý válec. Jsou spouštěny z kroku Step_Davkovani v hlavní rutině.

Způsob, kterým se kuličky dávkují, je čistě sekvenční a v těchto rutinách je SFC kód velmi přehledný a jednoduchý. Dávkovací rutina se skládá z šesti kroků (viz obrázek 3.7 a obrázek 3.8).

První inicializační krok je bod, ze kterého vede výběrová větev. Pokud je nastaven počet požadovaných kuliček na 0, rutina postupuje výběrovou větví do posledního kroku, kde čeká na ostatní.

Následující 4 kroky střídavě vypínají a zapínají západky tak, aby jedna kulička propadla za jeden cyklus programu. V třetím kroku Step_CounterL/M/R se pomocí vnitřního tagu Step.Count počítá, kolikrát se tento krok již spustil a právě podle této proměnné určujeme konec dávkování a přechod do posledního kroku. V opačném případě, pokud před posledním krokem nemá Count správnou hodnotu, cyklus se opakuje. Dávkování pro zbylé válce je vytvořeno stejně s jejich příslušnými proměnnými.

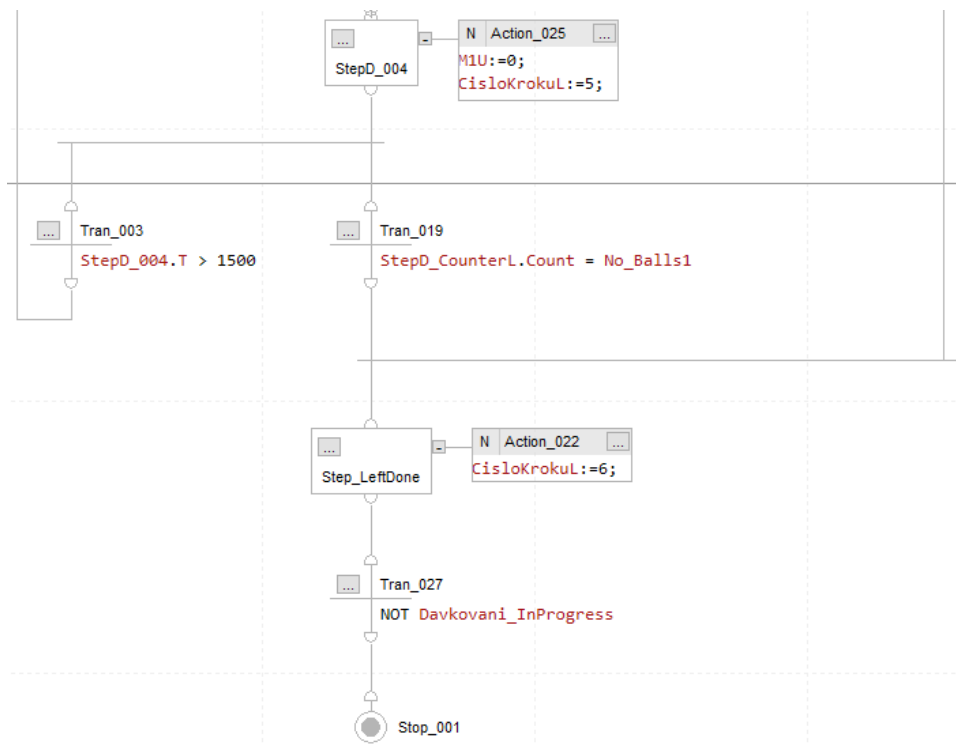


Obr. 3.7: Rutina dávkování pro levý válec – část 1/2

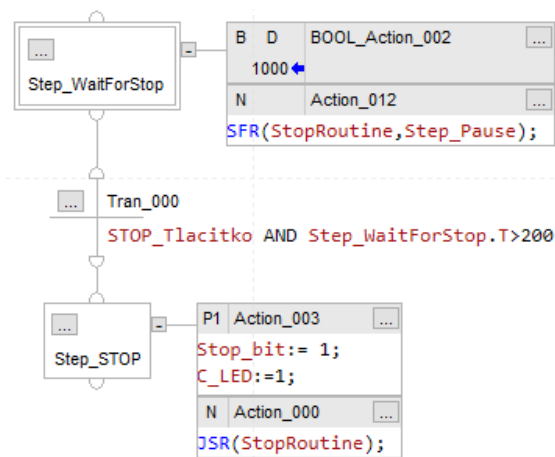
3.2.4 Konec programu a resetování programu na první krok

V předchozí kapitole byla zmíněna větev (network pro LAD), ve které se program ukončí. Tato větev běží souběžně se zbytkem programu, ale je až posledním krokem v ukončení programu. Hlavní část se váže na stav stop, ve kterém je program pozastaven a je uskutečněn v programu StopProgram.

Jsou zde dvě rutiny. Rutina DetekceStop slouží pro detekování sepnutí tlačítka STOP. Má dva kroky, přičemž první krok má dvě akce. Globálně definovaná booleanská akce zde slouží pro hlavní rutinu jako kontrola, zda-li nejsme ve stavu stop. Lze použít klasicky pouze proměnnou, do které zapíšeme hodnotu při vstupu do kroku. Takto lze booleanskou akci použít na dalších místech v programu. Druhá akce provádí reset rutiny StopRoutine.



Obr. 3.8: Rutina dávkování pro levý válec – část 2/2



Obr. 3.9: Rutina DetekceStop

Pokud uživatel sepně tlačítko STOP, tak se rutina posune na druhý krok, ve kterém se skočí do subrutiny StopRutina.

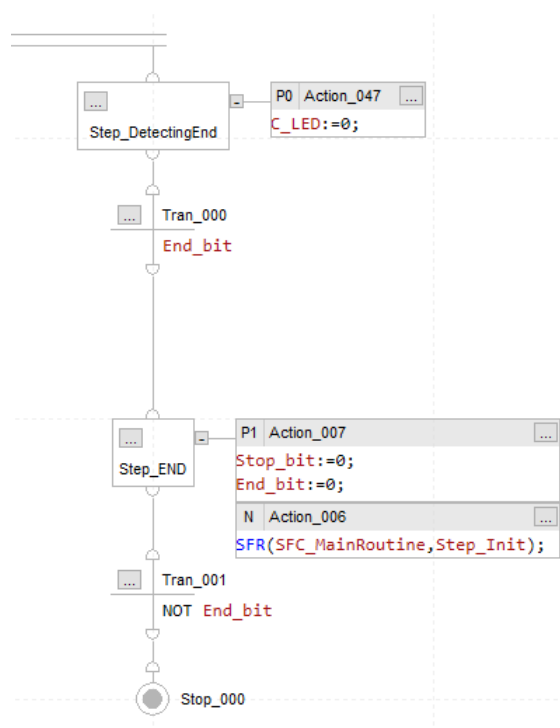
V prvním kroku subrutiny StopRutina je program pozastaven a čeká se na stisknutí STOP nebo START tlačítka. V prvním kroku jsou čtyři akce, dvě pro ovládní indikátorů a dvě pro ovládní informačního okna v HMI. V obou větvích je po stik-

nutí zpoždění 50 ms, které je účelně vytvořeno pro akce s kvalifikátorem P0. Tyto akce provádějí příkaz SFR a vracejí se do první rutiny DetekceStop. Bez odezvy se krok provedl v jednom skenu a nemusely se provést všechny akce Action_001 nebo Action_004.

Pokud je stisknut START, tak se rutina pomocí SFR vrátí do první rutiny DetekceStop a program pokračuje, kde skončil. Ve větvi pro start je znovu použita booleanská akce a následně se její aktivita bere v potaz v přechodu z kroku Step_005.

Při opětovném stisknutí tlačítka STOP, se program postupně ukončí. Uvolní se západky a nastaví se proměnná End_bit do 1.

Jakmile se nastaví, tak bude v hlavní rutině splněna podmínka pro paralelní větev, která je zmíněna na začátku této kapitoly.



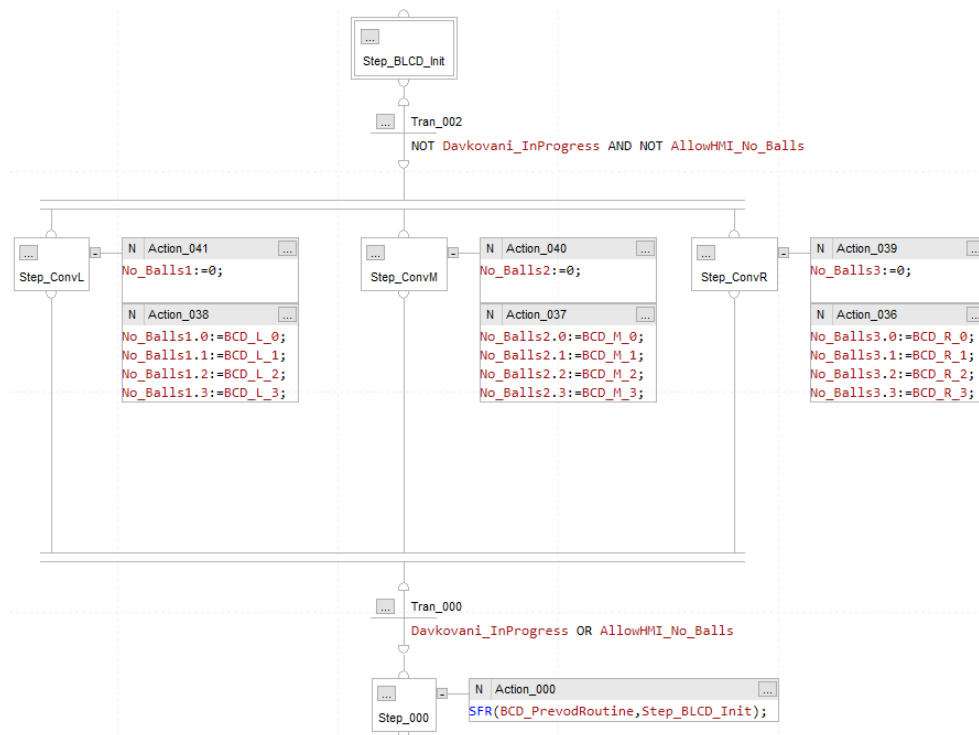
Obr. 3.11: Hlavní rutina a její paralelní větev pro ukončení programu

Po přechodu do kroku Step_END se program pomocí SFR vrátí zpět na inicializační krok a je možné znovu začít celý proces.

3.2.5 BCD_PrevodProgram

Každý číslicový volič má 4 binární výstupy, které dohromady tvoří číslo v BCD kódu. Bylo nutné je krátkým programem převést do proměnných No_Balls. Program se skládá z jedné rutiny, ve které zapisují příslušné výstupy do DINT proměnné.

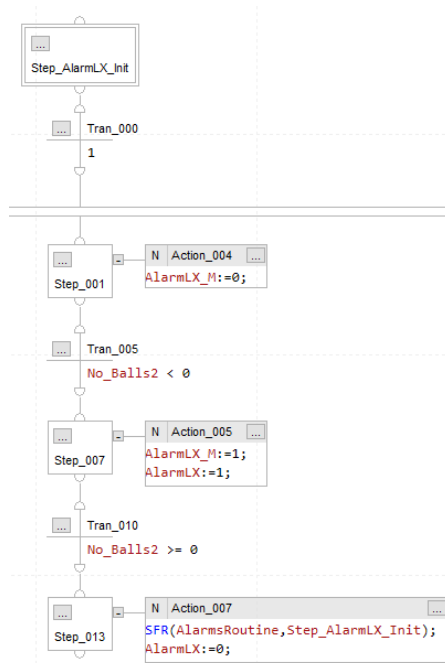
V rutině jsem použil konvergující větve, která spojuje všechny kroky do jednoho přechodu. Ten následně vede do kroku, který deaktivuje zápis do proměnných No_Balls, aby uživatel nebyl schopen změnit počet kuliček během dávkování.



Obr. 3.12: Rutina pro konverzi BCD kódu na DINT číslo

3.2.6 AlarmsProgram

Program slouží pro detekci překročení meze číslicového voliče a HMI vstupu. V tomto programu jsou definovány pro každý volič případy AlarmLX a AlarmHX. Samotná logika je zabudována jako podmínka v hlavní rutině.



Obr. 3.13: Detekce hodnoty nižší než nula pro Válec M

3.2.7 Informace_HMI_Logic

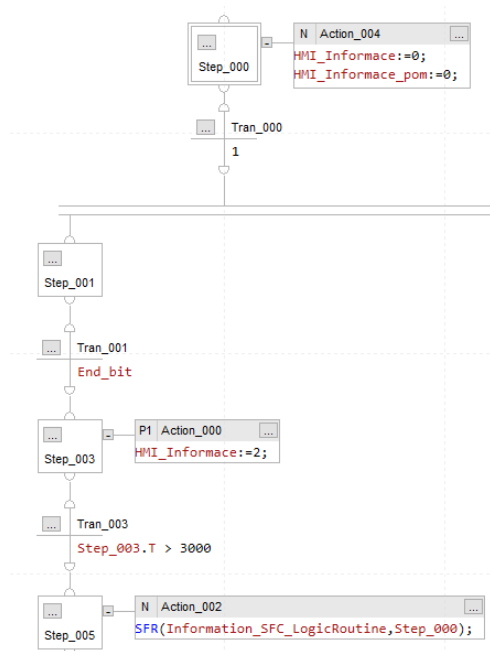
Slouží pouze pro informační okno v HMI a v tomto programu je definovaná jeho logika. Informační okno v HMI se zobrazí, pokud uživatel program pozastaví, ukončí nebo nechá pokračovat. Pro jednotlivé případy se zobrazí text, který uživateli potvrdí jeho voblu a po 3 sekundách zmizí.

Proměnná HMI_Informace je v HMI nastavena pro korespondující hodnotu v záložce Information settings.

3.2.8 TlacitkaSetupProgram

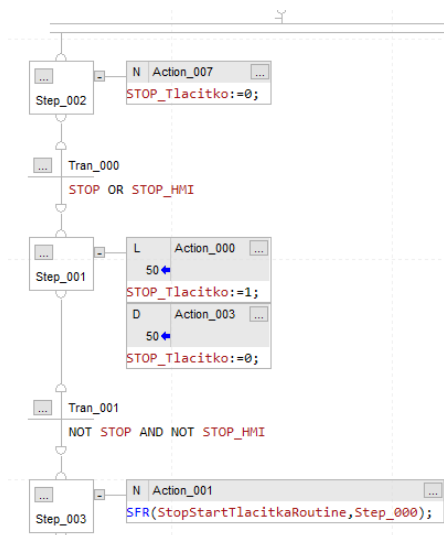
Program mění fungování tlačítek na fyzickém modelu a v HMI, aby uživatel po zmáčknutí tlačítka nemohl přeskočit několik kroků zároveň, převážně ze stavu STOP do hlavní rutiny.

V programu se vstupy tlačítek zapisují do nových Start_tlacitko a STOP_Tlacitko,



Obr. 3.14: Logika zobrazení informačního okno pro ukončení programu

které jsou po stiknutí v logické 1 na 50 ms a poté v logická 0 dokud uživatel znovu nestiskne tlačítko.



Obr. 3.15: Logika proměnných Start_tlacidko a Stop_tlacidko

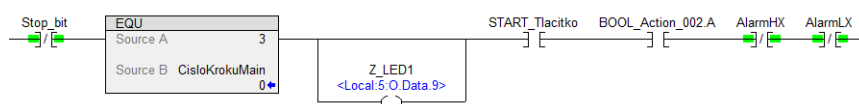
3.2.9 Porovnání hlavní rutiny v jazyce LAD

V příloze D jsou uvedeny verze hlavní rutiny v jazycích ST a LAD, které jsou k sobě identické, ale oproti SFC rutině mají menší rozdíly, protože některé části kódu se aplikují rozdílně oproti SFC. Rozdíl, který není z důvodu použití jazyku je, že ventilátory se spínají v SFC až v druhém kroku, ale to je pouze z důvodu práce s fyzickým modulem.

V LAD kódu je použit příkaz S:FS (S: First Scan - aktivní pouze při prvním startu programu), který v SFC nelze použít, protože i přesto, že každá akce používá jazyk ST, tak ne všechny funkce jsou pro něj podporovány.

Při programování SFC byly potíže pokud několik programů začalo reagovat na stejný vstup v jednom skenu, z tohoto důvodu byla vytvořena smyčka v kroku Step_005 a podmínka IF v kroku dávkování (viz obrázek 3.17). V LAD a ST tyto problémy jsou také, ale jejich řešení je mnohem elegantnější.

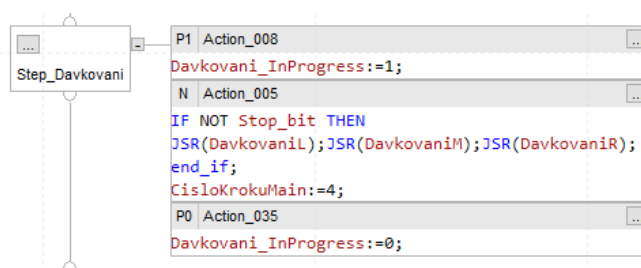
Krok STEP_ZLED, ve kterém je vytvořena smyčka v SFC kódu, je v LAD a ST jazyku vyřešena podmínkou Stop_bit na začátku networku. Další překážkou v SFC



Obr. 3.16: Porovnání kroků Step_ZLED a Step_ZLED2 v SFC s jazykem LAD

kódu byl proces dávkování, který je limitován podmínkou IF NOT Stop_bit.

Jak bylo řečeno v kapitole 3.2.3), tato podmínka obchází princip sekvenčního pro-



Obr. 3.17: Krok Step_Davkovani

gramování. Je použita stejně jako podmínka v LAD kódu.

Kód pro hlavní rutinu v jazyce LAD a ST je výrazně přehlednější oproti SFC. V hlavní rutině se provádí ukončení kódu v paralelní větvi. V SFC je vytvořena



Obr. 3.18: Krok Step_Davkovani v SFC v jazyku LAD

pro tuto funkci větev, která rozšiřuje celý kód, zatímco v jazyku LAD je přidáný network, který přehlednost kódu tolik neovlivňuje.

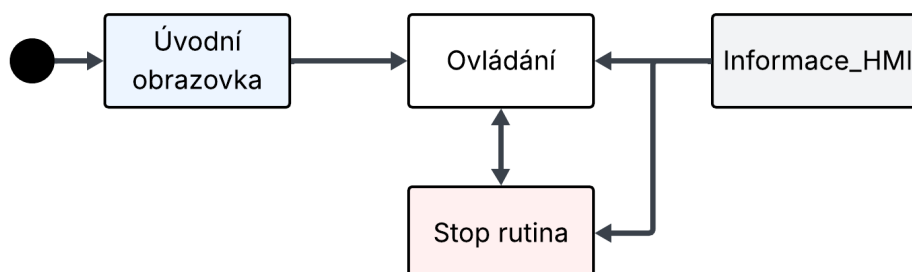
V jazyku SFC se automaticky tvoří spoje z přechodů do kroků a nelze s nimi posouvat tak, aby byl program přehledný. To znamená, že pokud je smyček v programu více, začnou se mezi sebou překrývat a kód se stává velmi špatně čitelný.

Vytváření a řešení paralelních procesů je výhodnější v jazyce LAD, ale pokud je nutné vytvořit sekvenční proces, tak SFC je výhodnější. Kód dávkování je velmi jednoduchý, ale pokud by byl vytvořen v LAD, bylo by nutné vytvořit několik časovačů a čítač. Všechny tyto funkce má každý krok již zabudované a velice zjednodušují sekvenční řízení.

3.3 Vizualizace demonstrační úlohy

Visualizace je vytvořena v aplikaci FactoryTalk View Studio ME verze 12.0.0. Skládá se z čtyř obrazovek. Zobrazuje uživateli, v jakém kroku se nachází a jak postupně jde celým programem.

První obrazovka je úvodní a po jejím přeskočení tlačítkem se zobrazí obrazovka Ovládání, ve které jsou diagramy programu a uživatel je schopen v této obrazovce projít celý program. Je zde možnost zobrazit obrazovku stop rutiny. Poslední obrazovka je informační a uživatel ji nemůže samostatně zobrazit. Zobrazí se po aktivaci jednoho z její prvků na spodní část HMI.

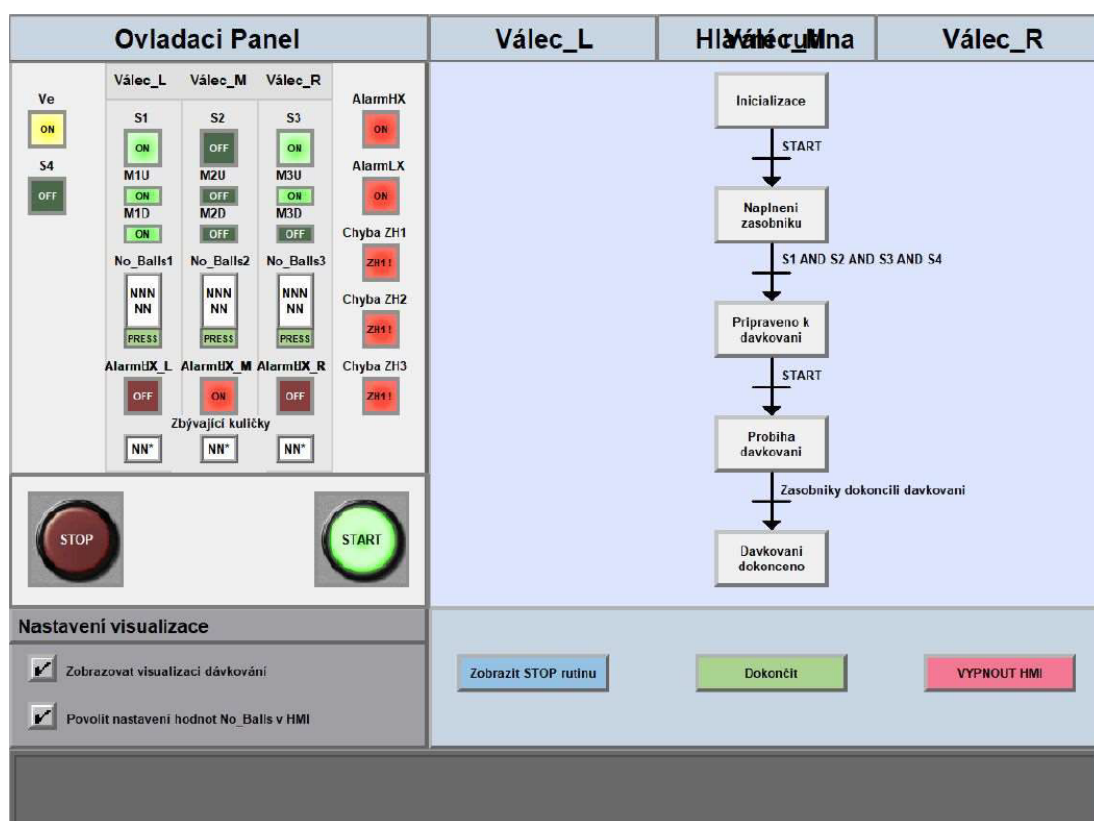


Obr. 3.19: Diagram obrazovek vizualizace

3.3.1 Obrazovka Ovládání

Tato ovládací obrazovka je hlavní člen vizualizace. Obrazovka je rozdělena na několik částí. Velká část indikátorů je na obrazovce viditelná pouze pokud se stanou aktivní, toto platí hlavně pro hlášení chyb.

V levé části je ovládací panel, ve kterém jsou všechny indikátory. Ve spodní části pravé strany se nachází nastavení, které slouží pro zapínání funkcí vizualizace. Na pravé straně je znázorněna vizualizace programu ve formě diagramu, který zobrazuje, v jakém kroku uživatel momentálně je. Poslední část obrazovky je určena pro tlačítka, které ovládají HMI. Ve vizualizaci je omezena viditelnost indikátorů, které

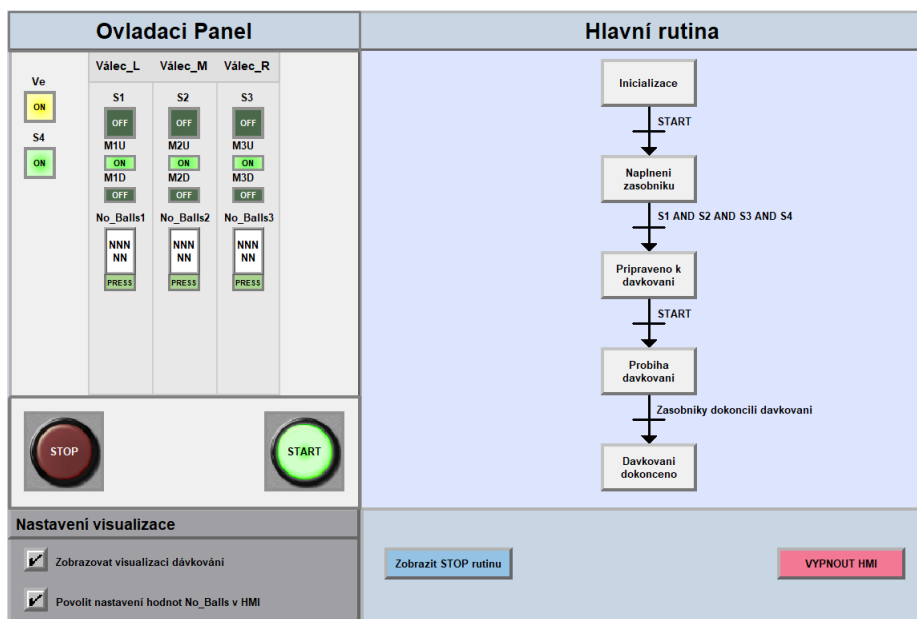


Obr. 3.20: Obrazovka ovládání s bez animací

nejdou relevantní v daný okamžik a uživatel po nahrání programu do PLC uvidí obrázek 3.21.

Ovládací panel

Každý válec má svůj vlastní sloupec, ve kterém jsou jeho příslušené indikátory. Pod zelenými indikátory se nachází číslcový vstup a indikátor, po jehož stisknutí může uživatel z HMI nastavit hodnotu No_Balls. Tato funkce se může zapnout/vypnout



Obr. 3.21: Obrazovka ovládání bez varovných indikátorů

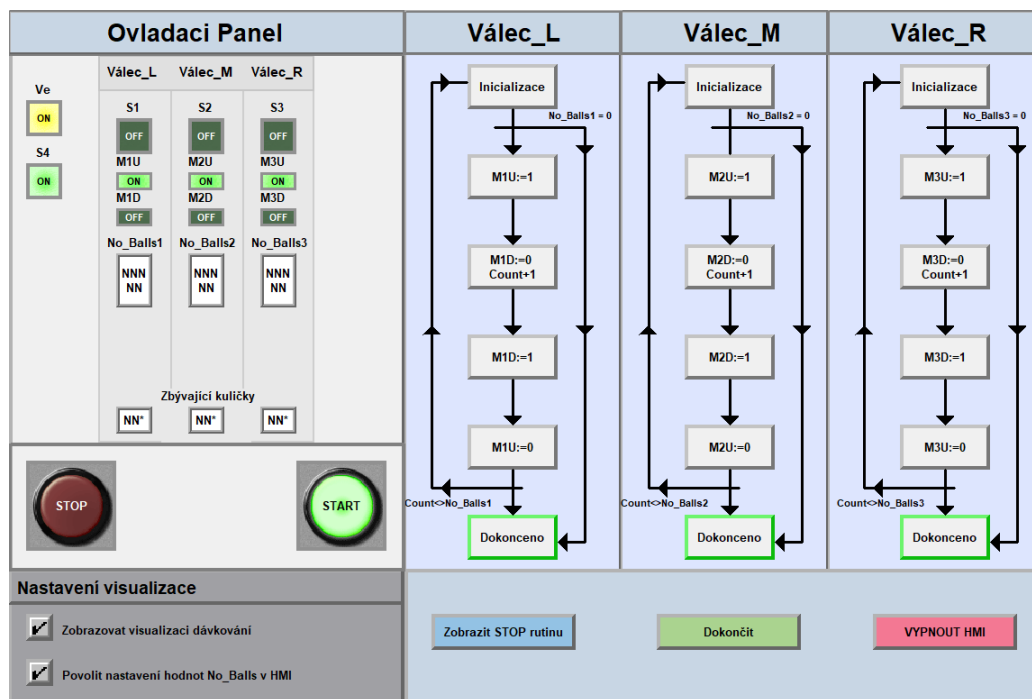
v panelu nastavení.

Červené indikátory se zobrazí pouze při příslušné chybě. Pokud se překročí meze No_Balls1 válce L, tak se zobrazí ve sloupci válce L chyba AlarmH/LX_L. Indikátory zbývajících kuliček se zobrazí po spuštění dávkování.

Panel vizualizace SFC

Vizualizace imituje vzhled SFC, pokud se nacházíme v kroku, krok bude zeleně ohraničený stejně jako v kódu. Tato sekce se mění v průběhu procesu. Nejdříve je zobrazena na obrazovce hlavní rutina, po spuštění dávkování se zobrazí dávkovací rutiny pro každý válec. Pokud uživatel nepotřebuje vizualizaci dávkování, může ji v nastavení vypnout a zůstane zobrazená pouze hlavní rutina.

Po dokončení dávkování se zobrazí zelené tlačítko „Dokončit“ po jehož kliknutí se potvrdí konec dávkování a vizualizace se přepne zpět na hlavní rutinu.



Obr. 3.22: Obrazovka ovládání po spuštění dávkování

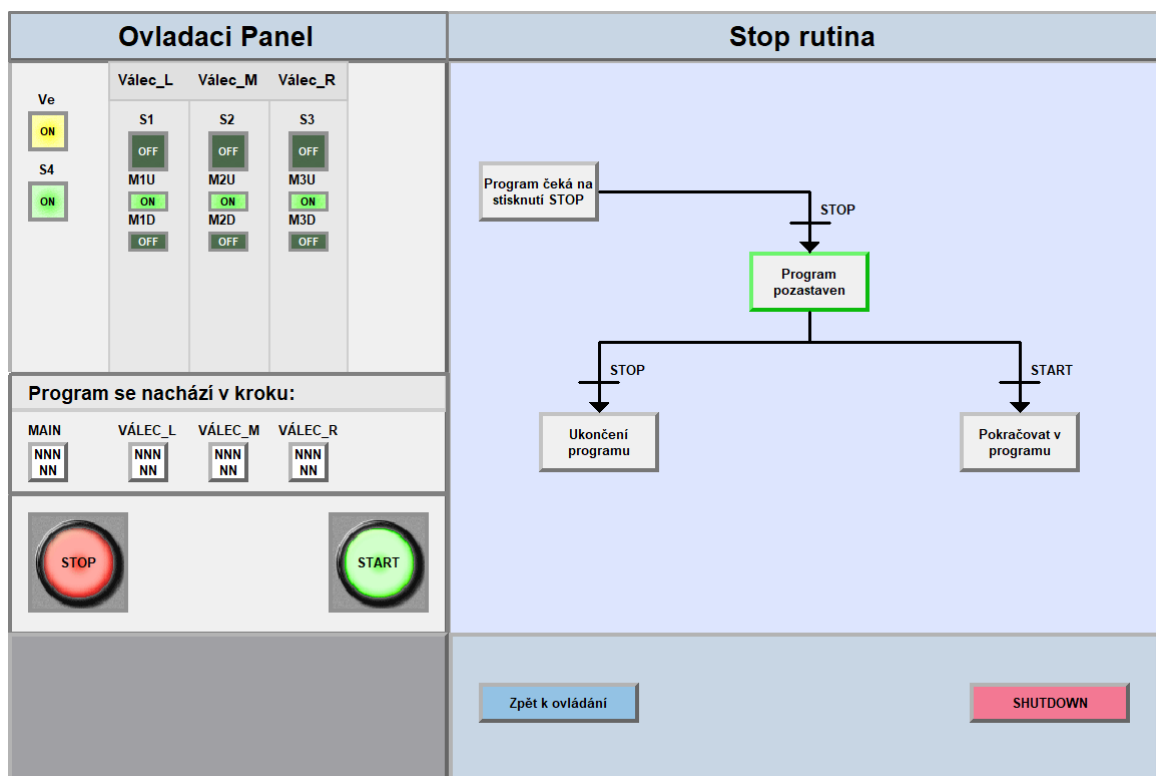
3.3.2 Obrazovka StopRutina

Na tuto obrazovku se dostaneme stiknutím modrého tlačítka na hlavní obrazovce vlevo dole. Zobrazí se vizualizace stop rutiny, kde můžeme vidět, zda-li jsme v zastaveném režimu nebo jestli program běží. Zároveň se na ovládacím panelu zobrazí kroky, ve kterých jsme zastavili v rutinách dávkování a hlavní rutiny.

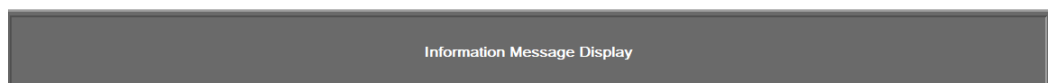
3.3.3 Obrazovka Informace_HMI

Do této obrazovky se uživatel přímo dostat nemůže, ale zobrazí se automaticky na spodním kraji obrazovek při specifických událostech. Události souvisí s pozastavením, ukončením a pokračováním programu a v jednotlivých případech se zobrazí text, který uživateli potvrdí jeho provedení.

Byla vytvořena, protože ukončení programu proběhne velmi rychle a nelze tento okamžik správně vizualizovat v diagramu.



Obr. 3.23: Obrazovka stop rutiny



Obr. 3.24: Informační obrazovka

Závěr

Cílem této bakalářské práce bylo vytvořit program v jazyce SFC pro laboratorní úlohu a zároveň navrhnout jeho vizualizaci v prostředí HMI.

V úvodní části byla provedena rešerše programovacích jazyků pro PLC. Dále se práce zaměřila na jazyk SFC, u kterého bylo zjištěno, že se výrazně liší od ostatních jazyků, zejména svou strukturou a přístupem k řízení. Byl popsán zjednodušený přehled základních principů SFC, přičemž zvláštní důraz byl kladen na kvalifikátory akcí, které tvoří klíčový prvek pro pochopení programování v tomto jazyce.

Následně byl jazyk SFC popsán na základních příkladech kódu, kde byly vysvětleny hlavní stavební bloky – Kroky a Akce – a jejich struktura včetně souvisejících tagů. Hlavní část této kapitoly byla věnována kvalifikátorům akcí, jejichž chování bylo demonstrováno na příkladech i trendech. Mezi nejpoužívanější kvalifikátory patří P1, P0 a N.

V praktické části byla popsána laboratorní úloha „Kuličky“ a vytvořen program v jazyce SFC podle zadání z předmětu BPC-PGA. Program byl rozdělen do šesti samostatných bloků kvůli paralelní povaze řízených procesů, kterou SFC neumožňuje přímo vyjádřit bez smyček nebo paralelních větví. V hlavním programu byly vytvořeny tři rutiny pro dávkování, protože v jazyce SFC není možné použít Add-On instrukce. V případě většího počtu válců by rozsah kódu negativně ovlivnil přehlednost a údržbu programu.

Pro porovnání byly v hlavním programu vytvořeny také rutiny v jazycích LAD a ST. Tyto jazyky umožňují přehlednější zápis paralelních procesů, což je jejich výraznou výhodou oproti jazyku SFC. V dalších částech programu byly naprogramovány pomocné funkce jako zastavení procesu, převod číslicového voliče a funkce pro komunikaci s HMI.

V závěru byla vytvořena vizualizace pro prostředí HMI, která obsahuje čtyři obrazovky, z nichž obrazovka „Ovládání je hlavní řídicí prvek vizualizace. Diagramy zobrazované na HMI znázorňují aktuální průběh programu a umožňují uživateli sledovat sekvenci kroků.

Z výsledného kódu vyplývá, že jazyk SFC je vhodný pro sekvenční řízení s menším počtem větví, kde přehledně zobrazuje tok programu. V případech složitější logiky s více paralelními procesy však může být použití jazyka SFC neefektivní a méně přehledné. V takových případech je vhodnější zvolit jiné jazyky.

Literatura

- [1] ŠTOHL, Radek. *BPC-PGA Programovatelné automaty* [online skripta]. Brno: VUT FEKT, 2024 [cit. 2024-12-10]. Dostupné z: <https://moodle.vut.cz/course/view.php?id=251123>.
- [2] CHHABRA, Atin. *What's the difference between a PLC and PAC?* [online]. [cit. 2024-10-12]. Schneider Electric Blog. Dostupné z: <https://blog.se.com/industry/machine-and-process-management/2018/12/24/whats-the-difference-between-a-plc-and-pac/>.
- [3] ČSN. ČSN EN 61131-3 ED.2, *Programovatelné řídicí jednotky – část 3: Programovací jazyky*. 2013
- [4] *Instruction list*. In: Wikipedia: the free encyclopedia [online]. San Francisco (CA): Wikimedia Foundation, 2024-[cit. 2025-02-04]. Dostupné z: https://en.wikipedia.org/wiki/Instruction_list.
- [5] *Logix 5000 Controllers General Instructions*. ROCKWELL AUTOMATION, INC. Milwaukee, 2024. [online]. Dostupné z: https://literature.rockwellautomation.com/idc/groups/literature/documents/rm/1756-rm003_-en-p.pdf.
- [6] *Logix 5000 Controllers Sequential Function Charts*. ROCKWELL AUTOMATION, INC. Milwaukee, 2024. [online]. Dostupné z: https://literature.rockwellautomation.com/idc/groups/literature/documents/pm/1756-pm006_-en-p.pdf.
- [7] BARACOS, Paul. *Grafcet Step-by-Step*. [online]. Famic Automation, 1992, s. 1. Dostupné z: https://www.researchgate.net/publication/243782363_Grafcet_step_by_step [cit. 2025-05-19].
- [8] BARKER, Mike, JAWAHAR RAWTANI, Steve, MACKAY, Steve. Introduction. In: BARKER, Mike, JAWAHAR RAWTANI, Steve, MACKAY, Steve, eds. *Practical Batch Process Management*. Newnes, 2005, s. 1–16. ISBN 9780750662772. Dostupné z: <https://doi.org/10.1016/B978-075066277-2/50001-3> [cit. 2025-05-19].

A Fotografie demostračního modelu: Kuličky



Obr. A.1: Demonstrační model: Kuličky

Obrázek A.1 byl vložen do nástroje ChatGPT společně s textem: Generate an image based on my photo. The style should be much cleaner. There are three glass tubes, each with an orange proximity sensor. There are also six small pistons with ventilation on top of them, and one box located beneath the tubes.

Výsledek je vygenerovaný obrázek 3.1.

B Obsah elektronické přílohy

```
/ .....kořenový adresář přiloženého archivu
├── DemonstracniProgramKod
│   └── DemonstraceSFC_Uloha_Sysel.acd
├── VizualizaceProgram
│   ├── DemonstraceSFC_HMI_Sysel.apa
│   └── Data_Vizualizace
├── VytisknuteReportyProgramu
│   ├── DemonstraceSFC_Uloha_Sysel.pdf
│   ├── DemonstraceSFC_HMI_Sysel.pdf
│   └── JednotliveRutiny
│       ├── SFC_MainRoutine.pdf
│       ├── DavkovaniL.pdf
│       ├── DavkovaniM.pdf
│       ├── DavkovaniR.pdf
│       ├── LAD_MainRoutine.pdf
│       ├── ST_MainRoutine.pdf
│       ├── DetekceStop.pdf
│       ├── StopRoutine.pdf
│       ├── BCD_ConvertRoutine.pdf
│       ├── AlarmsRoutine.pdf
│       ├── Informace_SFC_LogicRoutine.pdf
│       └── StopStartTlacitkaRoutine.pdf
```